DiscoGMPI

User Guide and Technical Reference

DiscoGMPI project

Version 0.1.0
June 2026

About this guide

This manual describes the `DiscoGMPI` software as implemented in the repository at the time of writing. It covers the mathematical model, discontinuous Galerkin (DG) discretization, Maxwell formulations, MPI decomposition, installation, mesh preparation, public workflows, examples, tests, benchmarks, diagnostics, and known limitations.

`DiscoGMPI` is the distributed-memory development line derived from DiscoG3D. It is a separate Julia package and repository. Work performed in `DiscoGMPI` does not overwrite the original DiscoG3D source tree.

Scope. The principal production path documented here is the source-free, three-dimensional Maxwell system on affine tetrahedral meshes, with explicit time integration. The MPI path supports piecewise-constant isotropic material properties, PEC, PMC, first-order absorbing, and translational periodic boundaries. It also supports the six-field nonlinear PML of Abarbanel, Gottlieb, and Hesthaven for dimensionless homogeneous vacuum problems. Impressed sources remain future work.

Notation. Superscripts $-$ and $+$ denote the local and neighboring traces at a face. The unit normal $\boldsymbol{n}$ points out of the minus element. The polynomial degree is denoted by N, the number of volume nodes per tetrahedron by N_p, and the number of nodes per triangular face by N_{fp}.

Contents

About this guide i

1 Overview 1
- 1.1 Purpose . . . 1
- 1.2 Repository layout . . . 1
- 1.3 Supported execution modes . . . 2

2 Mathematical Model 3
- 2.1 Source-free Maxwell equations . . . 3
- 2.2 Electromagnetic energy . . . 3
- 2.3 PEC boundary condition . . . 3
- 2.4 PMC and absorbing boundary conditions . . . 4
- 2.5 Six-field nonlinear PML . . . 4

3 Tetrahedral DG Discretization 7
- 3.1 Mesh and approximation space . . . 7
- 3.2 Affine mapping and physical derivatives . . . 7
- 3.3 Topology, traces, and lifting . . . 7
- 3.4 Hesthaven–Warburton formulation . . . 8
- 3.5 Poisson-bracket formulation . . . 8

4 Time Integration 10
- 4.1 Explicit Runge–Kutta methods . . . 10
- 4.2 Explicit partitioned symplectic methods . . . 10
- 4.3 CFL estimate . . . 11

5 Distributed-Memory Design 12
- 5.1 Element ownership and face halos . . . 12
- 5.2 Face orientation . . . 12
- 5.3 Maxwell communication . . . 12
- 5.4 Root-only mesh distribution . . . 13
- 5.5 Replicated constructor . . . 14

6 Installation and Environment 15
- 6.1 Prerequisites . . . 15
- 6.2 Instantiate the Julia environment . . . 15
- 6.3 Check MPI . . . 15
- 6.4 Thread configuration . . . 16

7 Mesh Preparation **17**
7.1 RawVTUMesh format . . . 17
7.2 Boundary and material tags . . . 17
7.3 Distributed periodic boundaries . . . 18
7.4 Inspect a VTU mesh . . . 18
7.5 Generate a METIS mesh . . . 18
7.6 Visualize a partition . . . 19
7.7 Mesh-quality requirements . . . 19

8 Programming with DiscoGMPI **20**
8.1 Construct a serial DG discretization . . . 20
8.2 Select the threaded backend . . . 20
8.3 Create an initial Maxwell field . . . 20
8.4 Run a serial Hesthaven–Warburton case . . . 21
8.5 Run a Poisson-bracket case . . . 21
8.6 Build and run an MPI case . . . 21
8.7 Distributed Poisson-bracket run . . . 22
8.7.1 Poisson-bracket run with nonlinear PML . . . 23
8.8 Energy diagnostics . . . 23
8.9 Global-to-local test initialization . . . 23

9 Running Included Examples **25**
9.1 Distributed Maxwell demonstration . . . 25
9.2 Distributed PEC, PMC, and nonlinear-PML cavity driver . . . 25
9.2.1 Diagnostics and production output . . . 26
9.3 Distributed periodic Poisson-bracket traveling wave . . . 26
9.3.1 Generating meshes with a configurable x resolution . . . 27
9.3.2 Automatic METIS partitioning . . . 28
9.3.3 Polynomial-order requirement . . . 29
9.3.4 Diagnostics and production output . . . 29
9.4 Distributed nonlinear PML pulse . . . 30
9.5 Empty-domain incident-wave PML gate . . . 30
9.6 PEC sphere boundary-residual gate . . . 31
9.7 Qualitative scattered E_x/H_y field gate . . . 32
9.8 Coarse PEC-sphere RCS gate . . . 32
9.9 Production Poisson-bracket PML validation . . . 34
9.10 Stationary PEC cube . . . 35
9.11 Stationary central-flux ESPRK case . . . 35
9.12 Stationary Poisson-bracket ESPRK case . . . 36
9.13 Upwind convergence study . . . 36
9.14 Central-flux partitioned convergence study . . . 36
9.15 Poisson-bracket threaded convergence study . . . 36
9.16 Distributed Poisson-bracket convergence study . . . 37
9.17 Distributed periodic plane-wave convergence study . . . 38

10 Radar Cross Section: Theory, Conventions, and Numerical Evaluation **40**
10.1 Purpose of RCS . . . 40
10.2 Physical Assumptions . . . 41
10.3 Time-Harmonic Convention . . . 41
10.4 Maxwell Scattering Problem . . . 42
10.4.1 PEC Boundary Condition . . . 42
10.4.2 Absorbing Layer . . . 42

10.5 Angular Coordinates . . . 43
10.6 Near-To-Far Transformation . . . 44
10.6.1 Outgoing Green Phase . . . 44
10.6.2 PEC Equivalent Surface Current . . . 44
10.6.3 Far-Field Reconstruction . . . 44
10.7 Time-Domain Fourier Extraction . . . 45
10.8 DG Surface Quadrature in the Code . . . 45
10.9 Algorithm Implemented in DiscoGMPI . . . 46
10.10 Observation Grid and CLI Controls . . . 47
10.11 Output Files . . . 47
10.12 Example RCS Figures . . . 48
10.13 MN5 EPW4/P4 RCS Validation Run . . . 51
10.14 NASA/Austin PEC Almond Benchmark . . . 51
10.14.1 Benchmark Definition . . . 54
10.14.2 Polarization and Normalization . . . 54
10.14.3 DiscoGMPI Almond Workflow . . . 55
10.14.4 Current Default Mesh . . . 56
10.14.5 Reference Comparison and Error Metrics . . . 57
10.14.6 Interpretation and Current Limitations . . . 57
10.14.7 Recommended Almond Commands . . . 57
10.15 Exact PEC-Sphere Mie Reference . . . 58
10.16 Validation Strategy . . . 59
10.16.1 Boundary Residual . . . 59
10.16.2 Qualitative Near Field . . . 59
10.16.3 Mie Comparison . . . 60
10.16.4 Austin Almond Reference Comparison . . . 60
10.17 Recommended Commands . . . 61
10.18 Common Failure Modes . . . 61
10.19 Checklist for a Quantitative RCS Study . . . 62
10.20 Code Map . . . 62
10.21 Summary . . . 63

11 Testing **69**
11.1 Package tests . . . 69
11.2 MPI face permutation test . . . 69
11.3 MPI METIS partition tests . . . 69
11.4 Distributed Maxwell equivalence tests . . . 69
11.5 Nonlinear PML tests . . . 70
11.6 Recommended pre-commit sequence . . . 70

12 Benchmarking **71**
12.1 Single benchmark . . . 71
12.2 Thread scaling sweep . . . 71
12.3 Plot scaling data . . . 71
12.4 MPI performance measurements . . . 72

13 Output and Visualization **73**
13.1 Serial VTU field output . . . 73
13.2 Partition output . . . 73
13.3 Distributed mesh files . . . 73
13.4 Checkpoint and restart . . . 74
13.5 ParaView time series . . . 74

13.6 Run I/O organization . . . 74
13.6.1 Automatic isolated run directories . . . 75
13.6.2 Configuration and input provenance . . . 75
13.6.3 Checkpoint paths . . . 76
13.7 Run and partition metadata . . . 76

14 Diagnostics and Troubleshooting **77**
14.1 MPI run hangs . . . 77
14.2 Wrong interface flux or rank-dependent error . . . 77
14.3 Missing or incorrect boundary conditions . . . 77
14.4 Periodic driver fails during partitioning . . . 78
14.5 Unstable time integration . . . 78
14.6 Poisson-bracket errors . . . 78
14.7 Threaded run is slower . . . 78
14.8 Memory use is larger than expected . . . 79

15 Current Limitations and Roadmap **80**
15.1 Current limitations . . . 80
15.2 Recommended next development stages . . . 81

A API Quick Reference **82**

B Command Quick Reference **85**

C Reproducibility Checklist **88**

License **89**

List of Tables

4.1 Explicit Runge–Kutta schemes available through `explicit_rk_scheme(order)`. 10

Chapter 1

Overview

1.1 Purpose

`DiscoGMPI` provides:

- nodal DG operators for affine tetrahedral meshes;
- source-free Maxwell volume and surface operators;
- Hesthaven–Warburton strong-form and Poisson-bracket formulations;
- central and upwind Maxwell fluxes where mathematically supported;
- serial and Julia-threaded local execution backends;
- explicit Runge–Kutta methods of orders one through five;
- explicit partitioned symplectic Runge–Kutta (ESPRK) schemes of orders one through six;
- METIS element partition input;
- owned-element and one-layer face-halo mesh construction;
- MPI face orientation and nodal permutation handling;
- nonblocking exchange of the six Maxwell traces;
- six-field nonlinear PML damping without auxiliary variables;
- distributed energy reductions and distributed time integration;
- partition visualization, regression tests, convergence cases, and backend benchmarks.

1.2 Repository layout

Path	Purpose
`src/DiscoGMPI.jl`	Package module, includes, and exported API.
`src/solver/`	Serial and threaded DG/Maxwell data structures and kernels.
`src/DistributedMesh3D.jl`	Ownership, ghost elements, local/global maps, face adjacency, and neighbor communication tables.

Path	Purpose
`src/DistributedDG.jl`	Rank-local DG construction, root-only mesh distribution, Maxwell trace exchange, distributed right-hand side, energy, and time integration.
`src/NonlinearPML.jl`	Nonlinear PML profiles, local source terms, and serial/distributed explicit RK wrappers.
`src/TraceMaps.jl`	General MPI trace maps and face-node permutation support.
`src/MetisIO.jl`	Legacy VTK tetrahedral reader and METIS mesh writer.
`src/PartitionVTK.jl`	Legacy VTK partition visualization writers.
`examples/solver/`	Serial and threaded Maxwell examples and convergence drivers inherited from DiscoG3D.
`examples/distributed_maxwell.jl`	End-to-end MPI Maxwell demonstration.
`examples/meshes/`	Sample VTK meshes, METIS partition files, and parameterized Gmsh geometries for structured and unstructured studies.
`benchmark/solver/`	Serial-versus-threaded right-hand-side benchmarks and plotting scripts.
`test/`	Package tests and explicitly launched MPI tests.

1.3 Supported execution modes

Serial
One Julia process using `SerialBackend()`.

Threaded
One Julia process using `ThreadedBackend()` and a Julia thread count selected with `-threads=N` or `JULIA_NUM_THREADS=N`.

Distributed memory
Multiple Julia processes launched with `mpiexec`. Every rank owns a subset of tetrahedra and stores a one-face halo.

Hybrid MPI plus threads
The distributed constructor accepts a backend, so a threaded local backend can be selected per rank. This path should be validated on the target machine before production use; the primary MPI regression tests use the serial local backend.

Chapter 2

Mathematical Model

2.1 Source-free Maxwell equations

For scalar, positive, elementwise-constant permittivity ε_K and permeability μ_K, `DiscoGMPI` advances on each tetrahedron K

$$\varepsilon_K \frac{\partial \boldsymbol{E}}{\partial t} = \nabla \times \boldsymbol{H}, \tag{2.1}$$

$$\mu_K \frac{\partial \boldsymbol{H}}{\partial t} = -\nabla \times \boldsymbol{E}. \tag{2.2}$$

The element wave speed, impedance, and admittance are

$$c_K = \frac{1}{\sqrt{\varepsilon_K \mu_K}}, \qquad Z_K = \sqrt{\frac{\mu_K}{\varepsilon_K}}, \qquad Y_K = \frac{1}{Z_K}. \tag{2.3}$$

The divergence constraints

$$\nabla \cdot (\varepsilon \boldsymbol{E}) = 0, \qquad \nabla \cdot (\mu \boldsymbol{H}) = 0 \tag{2.4}$$

are not solved as separate equations. Their preservation depends on the initial condition, mesh, spatial discretization, flux, and time integrator.

2.2 Electromagnetic energy

The continuous energy is

$$\mathcal{E}(t) = \frac{1}{2} \int_\Omega \left(\varepsilon |\boldsymbol{E}|^2 + \mu |\boldsymbol{H}|^2 \right) \, \mathrm{d}\boldsymbol{x}. \tag{2.5}$$

The code evaluates its DG approximation element by element using the reference mass matrix and affine Jacobian determinant:

$$\mathcal{E}_h = \frac{1}{2} \sum_K |J_K| \left[\varepsilon \sum_{\alpha \in \{x,y,z\}} \boldsymbol{E}_{\alpha,K}^T M \boldsymbol{E}_{\alpha,K} + \mu \sum_{\alpha \in \{x,y,z\}} \boldsymbol{H}_{\alpha,K}^T M \boldsymbol{H}_{\alpha,K} \right]. \tag{2.6}$$

In MPI runs only owned elements contribute locally, followed by an `MPI.Allreduce` sum. Ghost elements are therefore not double counted.

2.3 PEC boundary condition

The perfect-electric-conductor condition is

$$\boldsymbol{n} \times \boldsymbol{E} = \boldsymbol{0} \qquad \text{on } \partial\Omega_{\mathrm{PEC}}. \tag{2.7}$$

`DiscoGMPI` imposes it through a reflected exterior trace:

$$\boldsymbol{E}^{+} = -\boldsymbol{E}^{-} + 2(\boldsymbol{n}\cdot\boldsymbol{E}^{-})\boldsymbol{n}, \tag{2.8}$$

$$\boldsymbol{H}^{+} = \boldsymbol{H}^{-}. \tag{2.9}$$

This reverses the tangential electric field while retaining its normal component. Boundary-condition selection is tag based. A typical registry is:

```
registry = MaxwellBoundaryRegistry(
    Dict(10 => DiscoGMPI.MaxwellBC_PEC),
)
```

An unregistered boundary tag maps to `MaxwellBC_None`; it does not silently become PEC.

2.4 PMC and absorbing boundary conditions

The perfect-magnetic-conductor state reflects the tangential magnetic field and retains the electric trace. It is selected with `MaxwellBC_PMC`. The `MaxwellBC_Absorbing` state is a first-order, impedance-matched characteristic condition: the incoming tangential characteristic is set to zero using the adjacent element impedance. It is not a PML and is most accurate for locally normally incident waves.

PMC reflection is supported by both the Hesthaven–Warburton and Poisson-bracket surface operators, including distributed faces. For a PMC cavity validation, the production driver uses electromagnetic duality:

$$\boldsymbol{E}_{\mathrm{PMC}} = Z\boldsymbol{H}_{\mathrm{PEC}}, \qquad \boldsymbol{H}_{\mathrm{PMC}} = -\frac{1}{Z}\boldsymbol{E}_{\mathrm{PEC}}, \qquad Z = \sqrt{\mu/\varepsilon}.$$

```
registry = MaxwellBoundaryRegistry(Dict(
    10 => MaxwellBC_PEC,
    20 => MaxwellBC_PMC,
    30 => MaxwellBC_Absorbing,
))
```

2.5 Six-field nonlinear PML

`DiscoGMPI` implements the primary nonlinear PML proposed by Abarbanel, Gottlieb, and Hesthaven for time-dependent electromagnetics in three dimensions. The implementation modifies the six physical Maxwell fields directly and introduces no auxiliary unknowns. For positive isotropic, piecewise-constant material coefficients ε_K and μ_K, the implemented elementwise form is

$$\varepsilon_K\frac{\partial \boldsymbol{E}}{\partial t} = \nabla\times\boldsymbol{H} + (\boldsymbol{\sigma}\odot\boldsymbol{P}_H)\times\boldsymbol{H}, \tag{2.10}$$

$$\mu_K\frac{\partial \boldsymbol{H}}{\partial t} = -\nabla\times\boldsymbol{E} + (\boldsymbol{\sigma}\odot\boldsymbol{P}_E)\times\boldsymbol{E}, \tag{2.11}$$

where $\odot$ denotes componentwise multiplication,

$$\boldsymbol{P}_H = \frac{\boldsymbol{E}\times\boldsymbol{H}}{a\mu_K|\boldsymbol{H}|^2 + (1-a)\varepsilon_K|\boldsymbol{E}|^2 + \delta}, \tag{2.12}$$

$$\boldsymbol{P}_E = -\frac{\boldsymbol{E}\times\boldsymbol{H}}{a\varepsilon_K|\boldsymbol{E}|^2 + (1-a)\mu_K|\boldsymbol{H}|^2 + \delta}. \tag{2.13}$$

The recommended and default parameter is $a = 1/2$. The positive regularization δ, whose default is 10^{-12}, prevents division by zero where both fields vanish. Users should verify that the selected value is small relative to the squared field scale of their problem.

For nonnegative damping coefficients $\boldsymbol{\sigma} = (\sigma_x, \sigma_y, \sigma_z)$, the regularized PML contribution to the pointwise energy balance is

$$\frac{1}{2}\frac{\partial}{\partial t}\left(\varepsilon_K|\boldsymbol{E}|^2 + \mu_K|\boldsymbol{H}|^2\right) = -\nabla\cdot(\boldsymbol{E}\times\boldsymbol{H}) - \sum_{i=1}^{3}\sigma_i(\boldsymbol{E}\times\boldsymbol{H})_i^2\left(\frac{1}{D_H}+\frac{1}{D_E}\right), \tag{2.14}$$

with

$$D_H = a\mu_K|\boldsymbol{H}|^2 + (1-a)\varepsilon_K|\boldsymbol{E}|^2 + \delta, \qquad D_E = a\varepsilon_K|\boldsymbol{E}|^2 + (1-a)\mu_K|\boldsymbol{H}|^2 + \delta.$$

The local PML term is therefore nonpositive.

Sign convention. The magnetic source in equation (2.11) follows Appendix A and the energy identity in the 2006 paper. The sign printed in its vector equation (2.10) is inconsistent with both. This choice is covered by a component-level regression test and a discrete energy-decay test.

Reference. S. Abarbanel, D. Gottlieb, and J. S. Hesthaven, "Non-Linear PML Equations for Time Dependent Electromagnetics in Three Dimensions," *Journal of Scientific Computing*, vol. 28, nos. 2–3, 2006, DOI: 10.1007/s10915-006-9072-1.

Damping profiles. `polynomial_pml_sigma` constructs a one-sided profile that is zero at the interior interface and rises to `sigma_max` at the outer boundary:

$$\sigma(\xi) = \sigma_{\max}\left[\operatorname{clamp}\left(\frac{\xi-\xi_{\text{interface}}}{\xi_{\text{outer}}-\xi_{\text{interface}}}, 0, 1\right)\right]^m.$$

Reversing the interface and outer-boundary coordinates produces a layer on the opposite side. Independent functions may be supplied for σ_x, σ_y, and σ_z.

```
pml = build_maxwell_nonlinear_pml(
    distributed_dg;
    sigma_x = (x, y, z) -> polynomial_pml_sigma(
        x, 1.5, 2.0; sigma_max=12.0, degree=2
    ),
    a = 0.5,
    regularization = 1e-12,
)

scheme = explicit_partitioned_symplectic_rk_scheme(4; first_partition=:H)
work = MaxwellPartitionedRKWorkspace(U, scheme)
distributed_maxwell_nonlinear_pml_partitioned_symplectic_rk_step!(
    U, work, scheme, dt, distributed_dg, registry,
    PoissonBracketFormulation(MaxwellFlux_Alternating), pml;
    ε = 2.0,
    μ = 3.0,
)
```

The periodic wrapper is `distributed_periodic_maxwell_nonlinear_pml_partitioned_symplectic_rk_s`. On MPI ranks, PML sources are added only to owned elements and ghost residuals remain zero. Because no additional state variables or traces are introduced, the existing six-component halo exchange is sufficient. Explicit-RK PML helper functions remain available, but the production

Poisson-bracket drivers keep the requested ESPRK scheme when PML damping is active. The material-aware overloads accept `MaxwellElementMaterials`; the scalar overloads accept `epsilon`/`mu` through the usual Maxwell RHS coefficients.

Chapter 3

Tetrahedral DG Discretization

3.1 Mesh and approximation space

The physical domain is partitioned into affine tetrahedra K. On each element, every scalar field component is represented in

$$\mathbb{P}^N(K) = \{p(x, y, z) : \deg p \leq N\}, \tag{3.1}$$

with

$$N_p = \frac{(N+1)(N+2)(N+3)}{6} \tag{3.2}$$

volume nodes. A triangular face contains

$$N_{fp} = \frac{(N+1)(N+2)}{2} \tag{3.3}$$

nodes.

The reference tetrahedron has vertices

$$(-1,-1,-1), \quad (1,-1,-1), \quad (-1,1,-1), \quad (-1,-1,1).$$

The implementation builds equispaced nodal points, an orthonormal polynomial basis, a Vandermonde matrix, nodal derivative matrices, an exact polynomial mass matrix, and face mass/lifting operators.

3.2 Affine mapping and physical derivatives

For an affine map $\boldsymbol{x} = \boldsymbol{x}_K(r, s, t)$, let J_K denote its Jacobian. Reference gradients are transformed by

$$\nabla_{\boldsymbol{x}} u = J_K^{-T} \nabla_{(r,s,t)} u. \tag{3.4}$$

The physical derivative matrices D_x, D_y, D_z and weak derivative matrices

$$S_x = MD_x, \qquad S_y = MD_y, \qquad S_z = MD_z \tag{3.5}$$

are precomputed for each tetrahedron. The Poisson-bracket formulation also uses S_x^T, S_y^T, S_z^T.

3.3 Topology, traces, and lifting

Faces are identified by sorted triples of global vertex IDs. A face appearing once is a boundary face; a face appearing twice is an interior face. More than two adjacent tetrahedra is treated as a non-manifold error.

For every interior face, the DG trace map stores:

- minus and plus element IDs;
- local face IDs;
- face-node indices;
- the plus-to-minus nodal permutation;
- the outward minus-side normal and face area.

Surface residuals are projected through the face mass matrix and lifted into the element volume. Conceptually, a face correction $\boldsymbol{f}$ contributes

$$M_K^{-1} M_{f,K} \boldsymbol{f} \tag{3.6}$$

to the nodal right-hand side, with the physical face area and element Jacobian included by the implementation.

3.4 Hesthaven–Warburton formulation

The strong-form volume terms directly approximate

$$\dot{\boldsymbol{E}}_K = \frac{1}{\varepsilon}(\nabla_h \times \boldsymbol{H})_K + \text{surface correction}, \tag{3.7}$$

$$\dot{\boldsymbol{H}}_K = -\frac{1}{\mu}(\nabla_h \times \boldsymbol{E})_K + \text{surface correction}. \tag{3.8}$$

Let $[\![\boldsymbol{E}]\!] = \boldsymbol{E}^+ - \boldsymbol{E}^-$ and $[\![\boldsymbol{H}]\!] = \boldsymbol{H}^+ - \boldsymbol{H}^-$. The central correction used by the code is

$$\boldsymbol{f}_E^{\mathrm{c}} = \frac{1}{2}\boldsymbol{n} \times [\![\boldsymbol{H}]\!], \tag{3.9}$$

$$\boldsymbol{f}_H^{\mathrm{c}} = -\frac{1}{2}\boldsymbol{n} \times [\![\boldsymbol{E}]\!]. \tag{3.10}$$

The upwind flux adds tangential penalties:

$$\boldsymbol{f}_E^{up} = \boldsymbol{f}_E^{\mathrm{c}} - \frac{Y}{2}\boldsymbol{n} \times (\boldsymbol{n} \times [\![\boldsymbol{E}]\!]), \tag{3.11}$$

$$\boldsymbol{f}_H^{up} = \boldsymbol{f}_H^{\mathrm{c}} - \frac{Z}{2}\boldsymbol{n} \times (\boldsymbol{n} \times [\![\boldsymbol{H}]\!]). \tag{3.12}$$

Select the formulation with:

```
central = HesthavenWarburtonFormulation(MaxwellFlux_Central)
upwind  = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)
```

The central flux is nondissipative in the ideal compatible setting. The upwind flux introduces numerical dissipation and is normally preferred when robust damping of unresolved jumps is more important than exact semidiscrete energy conservation.

3.5 Poisson-bracket formulation

The Poisson-bracket implementation is a partitioned weak-operator discretization. The electric update uses weak derivative matrices S_x, S_y, S_z, while the magnetic update uses their transposes. This pairing constructs a discrete skew structure and supports energy-compatible partitioned time stepping.

For an interior face, the implemented corrections can be summarized as

$$\boldsymbol{f}_E^{\mathrm{PB}} = -\frac{1}{2\varepsilon}\boldsymbol{n} \times (\boldsymbol{H}^- - \boldsymbol{H}^+), \tag{3.13}$$

$$\boldsymbol{f}_H^{\mathrm{PB}} = -\frac{1}{2\mu}\boldsymbol{n} \times (\boldsymbol{E}^- + \boldsymbol{E}^+). \tag{3.14}$$

Only the central flux is supported:

```
formulation = PoissonBracketFormulation()
```

Attempting to combine the Poisson-bracket formulation with the upwind flux raises an error because the upwind penalty is dissipative and does not define the implemented partitioned Poisson operator.

Chapter 4

Time Integration

4.1 Explicit Runge–Kutta methods

For a semidiscrete system $\dot{U} = R(U)$, an explicit Runge–Kutta method uses

$$U^{(i)} = U^n + \Delta t \sum_{j<i} a_{ij} K_j, \tag{4.1}$$

$$K_i = R(U^{(i)}), \tag{4.2}$$

$$U^{n+1} = U^n + \Delta t \sum_i b_i K_i. \tag{4.3}$$

Table 4.1: Explicit Runge–Kutta schemes available through `explicit_rk_scheme(order)`.

Order	Scheme	Stages
1	Forward Euler	1
2	Explicit midpoint	2
3	Classical third order	3
4	Classical fourth order	4
5	Dormand–Prince fifth-order update	7

The Dormand–Prince implementation uses the fifth-order update only. Adaptive step-size control and the embedded fourth-order estimator are not exposed.

4.2 Explicit partitioned symplectic methods

The Maxwell state is divided into electric and magnetic partitions. At every ESPRK stage, one partition is updated using its weight, the right-hand side is recomputed, and the complementary partition is updated using its own weight. Orders one through six are available:

```
scheme = explicit_partitioned_symplectic_rk_scheme(
    4;
    first_partition = :H,
)
```

For `PoissonBracketFormulation`, the magnetic partition must be advanced first:

```
formulation = PoissonBracketFormulation()

run_maxwell_partitioned_symplectic_time_steps!(
    U,
```

```
    dg,
    registry,
    formulation;
    psrk_order = 4,
    first_partition = :H,
    dt = dt,
    nsteps = nsteps,
)
```

Using the conventional explicit Runge–Kutta driver with the Poisson-bracket formulation is rejected by the high-level API.

4.3 CFL estimate

The code defines the element characteristic length

$$h_K = \frac{3V_K}{A_K}, \tag{4.4}$$

where V_K is the tetrahedron volume and A_K its total surface area. The recommended estimate is

$$\Delta t = \mathrm{CFL}\frac{\min_K h_K}{(2N+1)c}. \tag{4.5}$$

It is obtained with:

```
dt, sizes = estimate_maxwell_dt(
    mesh,
    dg.geometry,
    dg.ref;
    CFL = 0.10,
)
```

This expression is an estimate, not a proof of stability for every mesh, formulation, flux, and RK scheme. Production studies should include a step-size sensitivity check.

Chapter 5

Distributed-Memory Design

5.1 Element ownership and face halos

An element partition vector assigns every global tetrahedron to a zero-based MPI rank:

$$p_e \in \{0, \ldots, P-1\}. \tag{5.1}$$

For rank r, `DiscoGMPI` stores:

- every element for which $p_e = r$ as *owned*;
- every off-rank tetrahedron sharing a complete face with an owned element as a *ghost*;
- the nodes required by those owned and ghost elements;
- global-to-local and local-to-global maps;
- ownership, material IDs, face neighbors, volumes, and barycenters;
- one communication record per neighboring rank.

Only a one-face halo is needed because the Maxwell DG surface operator couples elements only through face traces.

5.2 Face orientation

A triangular face admits six vertex permutations. For each inter-rank face, the distributed mesh records the orientation code that maps the neighbor ordering to the local ordering. The trace layer converts face nodes to barycentric coordinates and precomputes one nodal permutation for each of the six orientations. Received plus traces are therefore presented in the same physical point ordering as the local minus trace.

This permutation step is essential for polynomial order $N > 1$: matching only the three vertices is insufficient because edge and interior face nodes must also be reordered.

5.3 Maxwell communication

At each right-hand-side evaluation:

1. The six owned face traces $(E_x, E_y, E_z, H_x, H_y, H_z)$ are packed for every neighbor.
2. All nonblocking receives are posted.

3. All nonblocking sends are posted.
4. `MPI.Waitall` completes the exchange.
5. Received values are unpacked into the corresponding ghost face nodes.
6. The local DG right-hand side is evaluated.
7. Every ghost-element right-hand-side column is set to zero, ensuring that time integration updates owned elements only.

For a neighbor with F_{rn} interface faces, the message contains

$$6N_{fp}F_{rn} \tag{5.2}$$

double-precision values per right-hand-side evaluation.

Current computation model. The local DG kernel currently evaluates owned and stored ghost elements before zeroing ghost residuals. Communication volume is already trace-only, but a future optimization can avoid unnecessary ghost volume work and overlap communication with owned volume calculations.

5.4 Root-only mesh distribution

The scalable constructor keeps the global mesh and partition vector only on a designated root rank. Root builds each rank's owned-plus-halo package and sends that serialized package to the destination. Non-root ranks never construct or retain the complete global mesh.

```
using MPI
using DiscoGMPI

MPI.Init()
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

root_mesh = rank == 0 ? read_vtu_mesh("mesh.vtu") : nothing
root_partition = if rank == 0
    read_metis_epart("mesh.mesh.epart.$nranks")
else
    nothing
end

ddg = build_distributed_dg_from_root(
    root_mesh,
    root_partition,
    2;
    comm = comm,
    root = 0,
)

MPI.Barrier(comm)
MPI.Finalize()
```

The file-based overload performs both reads only on root:

```
ddg = build_distributed_dg_from_root(
    "mesh.vtu",
    "mesh.mesh.epart.4",
    2;
    comm = MPI.COMM_WORLD,
)
```

All ranks must call the constructor collectively. Root validation errors are broadcast so that malformed input fails collectively rather than leaving worker ranks blocked.

5.5 Replicated constructor

When a global `RawVTUMesh` and partition vector already exist on every rank, the compatibility constructor remains available:

```
ddg = build_distributed_dg(
    global_mesh,
    elem_to_rank,
    order;
    comm = MPI.COMM_WORLD,
)
```

The root-only constructor is preferable for memory scaling on non-root ranks. Root still holds the complete input mesh during distribution.

Chapter 6

Installation and Environment

6.1 Prerequisites

- Julia 1.9 or later.
- An MPI implementation compatible with the MPI library selected by MPI.jl.
- An `mpiexec` command for multi-process runs.
- METIS and its `mpmetis` executable when generating new partitions.
- ParaView or another VTK viewer for visualization.
- Optional: Python 3 and Matplotlib for benchmark plots.
- Optional: a LaTeX installation to build this manual.

6.2 Instantiate the Julia environment

Run all commands from the repository root:

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
julia --project=. -e 'using DiscoGMPI; println("DiscoGMPI loaded")'
```

The package dependencies are recorded in `Project.toml` and `Manifest.toml`. Important dependencies include MPI.jl, ReadVTK.jl, VTKBase.jl, WriteVTK.jl, BenchmarkTools.jl, and Julia's LinearAlgebra standard library.

6.3 Check MPI

```
mpiexec -n 2 julia --project=. -e '
using MPI
MPI.Init()
println("rank ", MPI.Comm_rank(MPI.COMM_WORLD),
        " of ", MPI.Comm_size(MPI.COMM_WORLD))
MPI.Barrier(MPI.COMM_WORLD)
MPI.Finalize()
'
```

If this command fails before `DiscoGMPI` is loaded, fix the MPI.jl/runtime configuration first. In particular, do not mix an arbitrary system `mpiexec` with a different MPI library selected by MPI.jl.

6.4 Thread configuration

Thread counts are selected when Julia starts:

```
julia --project=. --threads=8 my_case.jl
JULIA_NUM_THREADS=8 julia --project=. my_case.jl
```

Inside Julia, verify the setting with:

```
println(Threads.nthreads())
```

Chapter 7

Mesh Preparation

7.1 RawVTUMesh format

The primary solver mesh container is:

```
RawVTUMesh(
    points,        # 3 x number_of_points, Float64
    tets,          # 4 x number_of_tetrahedra, Int
    tris,          # 3 x number_of_boundary_triangles, Int
    tet_cell_ids,
    tri_cell_ids,
    cell_data,
)
```

Connectivity uses Julia's one-based indexing. The XML VTU reader accepts tetrahedral volume cells and triangular surface cells and ignores unsupported cell types.

7.2 Boundary and material tags

By default, `DGDiscretization` looks for cell data named `boundary_id`. The array is indexed by the original VTK cell IDs; `tri_data(mesh, "boundary_id")` extracts its surface-triangle values.

The distributed constructors also accept:

```
boundary_tag_name = "boundary_id"
material_tag_name = "region_id"
```

Material IDs are preserved in the distributed topology and can be converted to element coefficients:

```
materials = maxwell_element_materials(
    distributed_dg,
    Dict(
        1 => MaxwellMaterial(1.0, 1.0),
        2 => MaxwellMaterial(4.0, 2.0),
    ),
)
```

The material-aware Maxwell overloads accept `materials` as a positional argument. Central fluxes use the coefficient of the updated element. The Hesthaven–Warburton upwind interface solver uses side-dependent impedance and admittance. Coefficients are currently isotropic and constant inside each tetrahedron.

7.3 Distributed periodic boundaries

`build_distributed_periodic_maxwell_exchange` gathers only owned periodic boundary descriptors, pairs shifted centroids and DG face nodes, and constructs directed rank-local faces. Remote periodic traces use a separate nonblocking MPI exchange; same-rank partners are read directly.

```
periodic = build_distributed_periodic_maxwell_exchange(
    distributed_dg,
    default_unit_box_periodic_specs();
    materials = materials,
)
```

Periodic boundary tags must map to `MaxwellBC_None` in the ordinary boundary registry. When material-aware periodic fluxes are used, pass `materials` while constructing the exchange so remote partner coefficients are recorded.

7.4 Inspect a VTU mesh

```
julia --project=. examples/solver/read_vtu_mesh.jl path/to/mesh.vtu
```

The inspection example prints mesh dimensions, connectivity, tags, topology, geometry, mappings, basis diagnostics, face operators, trace checks, flux-face checks, and several local operator tests. It is intentionally verbose and is a useful first check for a new mesh.

7.5 Generate a METIS mesh

The sample workflow begins with a legacy ASCII VTK tetrahedral mesh:

```
julia --project=. examples/write_metis.jl
```

This calls:

```
write_metis_mesh_from_vtk(
    "examples/meshes/tet_mesh.vtk",
    "examples/meshes/tet_mesh.mesh",
)
```

The generated METIS mesh contains one tetrahedron per line. Create P partitions with:

```
mpmetis examples/meshes/tet_mesh.mesh 4
```

METIS normally writes:

- `tet_mesh.mesh.epart.4`: one element partition ID per line;
- `tet_mesh.mesh.npart.4`: one node partition ID per line.

`DiscoGMPI` reads the element partition file. Partition IDs are expected to be zero based. For one-based files use:

```
parts = read_metis_epart(path; one_based_parts = true)
```

7.6 Visualize a partition

Edit `NPROCS` in `examples/write_partition_vtk.jl` if required, then run:

```
julia --project=. examples/write_partition_vtk.jl
```

The script writes:

- one global legacy VTK file with `partition_id` cell data;
- one owned-element VTK file per partition.

The principal visualization file is

`examples/visualization/partitioned_mesh.vtk`.

Open it in ParaView, color by `partition_id`, and select "Surface With Edges".

7.7 Mesh-quality requirements

- Tetrahedra must have four valid, distinct nodes.
- Every interior face must be shared by exactly two tetrahedra.
- Boundary triangles must match tetrahedral exterior faces.
- Volumes and face areas must be positive.
- The partition vector length must equal the tetrahedron count.
- Every partition ID must lie in `0:nranks-1`.
- Every MPI rank should normally own at least one element.

Chapter 8

Programming with DiscoGMPI

8.1 Construct a serial DG discretization

```
using DiscoGMPI

mesh = read_vtu_mesh("mesh.vtu")
dg = DGDiscretization(
    mesh,
    3;
    boundary_tag_name = "boundary_id",
    trace_tol = 1e-10,
    backend = SerialBackend(),
)
```

The resulting `DGDiscretization` contains the raw mesh, topology, geometry, affine mappings, reference element, face operators, physical operators, trace maps, flux faces, and backend.

8.2 Select the threaded backend

```
dg = DGDiscretization(
    mesh,
    3;
    backend = ThreadedBackend(),
)
```

The threaded kernels use face coloring so concurrently processed faces do not write to the same element residual.

8.3 Create an initial Maxwell field

```
Efun = (x, y, z) -> (
    sin(pi * x),
    0.0,
    sin(pi * z),
)

Hfun = (x, y, z) -> (
    0.0,
    cos(pi * y),
    0.0,
```

```
)

U = interpolate_maxwell_field(mesh, dg.ref, Efun, Hfun)
```

Each component is stored as an `Np x Ne` matrix. Columns are elements and rows are element-local DG nodes.

8.4 Run a serial Hesthaven–Warburton case

```
registry = MaxwellBoundaryRegistry(
    Dict(10 => DiscoGMPI.MaxwellBC_PEC),
)
formulation = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)

dt, _ = estimate_maxwell_dt(
    mesh,
    dg.geometry,
    dg.ref;
    CFL = 0.10,
)

run_maxwell_time_steps!(
    U,
    dg,
    registry,
    formulation;
    rk_order = 4,
    dt = dt,
    nsteps = 100,
    energy_every = 10,
)
```

8.5 Run a Poisson-bracket case

```
formulation = PoissonBracketFormulation()

run_maxwell_partitioned_symplectic_time_steps!(
    U,
    dg,
    registry,
    formulation;
    psrk_order = 4,
    first_partition = :H,
    dt = dt,
    nsteps = 100,
    energy_every = 10,
)
```

8.6 Build and run an MPI case

```
using MPI
```

```
using DiscoGMPI

MPI.Init()
comm = MPI.COMM_WORLD
rank = MPI.Comm_rank(comm)
nranks = MPI.Comm_size(comm)

root_mesh = rank == 0 ? read_vtu_mesh("mesh.vtu") : nothing
root_parts = rank == 0 ?
    read_metis_epart("mesh.mesh.epart.$nranks") :
    nothing

ddg = build_distributed_dg_from_root(
    root_mesh,
    root_parts,
    2;
    comm = comm,
)

Efun = (x, y, z) -> (sin(pi*x), 0.0, sin(pi*z))
Hfun = (x, y, z) -> (0.0, cos(pi*y), 0.0)
U = interpolate_maxwell_field(ddg, Efun, Hfun)

registry = MaxwellBoundaryRegistry(
    Dict(10 => DiscoGMPI.MaxwellBC_PEC),
)
formulation = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)

run_distributed_maxwell_time_steps!(
    U,
    ddg,
    registry,
    formulation;
    rk_order = 3,
    dt = 1e-4,
    nsteps = 100,
    energy_every = 10,
)

MPI.Barrier(comm)
MPI.Finalize()
```

Launch it with:

```
mpiexec -n 4 julia --project=. my_distributed_case.jl
```

8.7 Distributed Poisson-bracket run

```
run_distributed_maxwell_partitioned_symplectic_time_steps!(
    U,
    ddg,
    registry,
    PoissonBracketFormulation();
    psrk_order = 4,
    first_partition = :H,
    dt = 1e-4,
```

```
    nsteps = 100,
    energy_every = 10,
)
```

8.7.1 Poisson-bracket run with nonlinear PML

The nonlinear source couples the electric and magnetic partitions, so this case uses ordinary explicit RK:

```
pml = build_maxwell_nonlinear_pml(
    ddg;
    sigma_x = (x, y, z) -> polynomial_pml_sigma(
        x, 0.8, 1.0; sigma_max=12.0, degree=2
    ),
)

run_distributed_maxwell_nonlinear_pml_time_steps!(
    U,
    ddg,
    registry,
    PoissonBracketFormulation(),
    pml;
    rk_order = 4,
    dt = 1e-4,
    nsteps = 100,
    energy_every = 10,
)
```

8.8 Energy diagnostics

```
serial_energy = maxwell_energy(
    U,
    dg.ref,
    dg.mappings,
)

global_energy = distributed_maxwell_energy(
    U,
    ddg,
)

println(global_energy.electric)
println(global_energy.magnetic)
println(global_energy.total)
```

Distributed energy is collective: every rank must call it in the same order.

8.9 Global-to-local test initialization

`localize_maxwell_field` extracts owned and halo columns from a global field:

```
local_U = localize_maxwell_field(global_U, ddg)
```

This function is primarily useful for verification against a serial reference. For scalable production initialization, use `interpolate_maxwell_field(ddg, Efun, Hfun)` so no global field is required on worker ranks.

Chapter 9

Running Included Examples

9.1 Distributed Maxwell demonstration

```
mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl
```

The script:

- builds a PEC cube only on rank zero;
- partitions its tetrahedra round-robin;
- distributes owned elements and face halos;
- initializes a cavity field locally;
- uses the Hesthaven–Warburton upwind formulation;
- advances two RK3 steps;
- reports ownership, neighbors, and global energy.

9.2 Distributed PEC, PMC, and nonlinear-PML cavity driver

The production driver `examples/distributed_poisson_bracket_maxwell.jl` uses PEC boundaries by default. Select PMC walls with:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc
```

Without PML, the driver uses the requested H-first ESPRK method and compares against the corresponding PEC or dual PMC cavity eigenmode.

Add a polynomial nonlinear PML at all six coordinate bounds with:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc \
  --pml-width=0.2 \
  --pml-sigma-max=12 \
  --pml-degree=2 \
  --esprk-order=4
```

Both `-pml-width` and `-pml-sigma-max` must be positive to enable the PML, or both zero to disable it. The width must be smaller than half the shortest global domain extent. PML runs support scalar `-epsilon` and `-mu` values and keep the requested H-first ESPRK method. The cavity mode remains in the output as an undamped reference; it is not an exact solution of the damped problem.

9.2.1 Diagnostics and production output

The default output directory is `output/distributed_poisson_bracket/`. The driver writes the same production artifact family as the periodic driver:

- `diagnostics/energy.csv`: DG mass-matrix electric, magnetic, and total energy;
- `diagnostics/quadrature_diagnostics.csv`: independent-cubature field and energy errors, optical chirality, electric and magnetic charge, and linear and angular momentum;
- `fields.pvd` and `fields/fields_stepNNNNNNNN.pvtu`: ParaView time series;
- `diagnostics/integration_points_rankNNNN.csv`: final integration-point numerical and analytical values, including pointwise chirality;
- `config/run_metadata.toml`, `config/resolved_config.toml`, and `config/partition_metadata.csv`;
- `input/input_manifest.toml`;
- `checkpoints/stepNNNNNNNN/`: checksummed rank-local restart data;
- `status.toml`: current run status.

For backward compatibility, the run root also contains symlinks named `energy.csv`, `quadrature_diagnostics.csv`, `run_metadata.toml`, and `partition_metadata.csv`.

At every cubature point, the driver evaluates

$$\chi = \frac{1}{2}\left(\varepsilon\, \boldsymbol{E}\cdot\nabla\times\boldsymbol{E} + \mu\, \boldsymbol{H}\cdot\nabla\times\boldsymbol{H}\right).$$

The numerical curls use the physical element derivative operators. The exact cavity curls are evaluated analytically for both PEC and dual PMC modes. In PML runs the same undamped cavity field remains the diagnostic reference, so field, energy, and chirality errors should be interpreted as reference errors, not as exact damped-solution errors.

The ParaView output writes each owned DG element as a VTK Lagrange tetrahedron containing every DG node. The rank-local VTU pieces include `ElectricField`, `MagneticField`, `ExactElectricField`, `ExactMagneticField`, `ElectricFieldError`, and `MagneticFieldError`, plus field magnitudes, global element IDs, owner rank, polynomial order, and physical time.

9.3 Distributed periodic Poisson-bracket traveling wave

The production periodic driver is:

```
examples/distributed_periodic_poisson_bracket_maxwell.jl
```

By default it uses the legacy VTK mesh `examples/meshes/periodic_box_2x1x1_nx4.vtk`, whose domain is

$$\Omega = [0,2]\times[0,1]\times[0,1].$$

The mesh contains four structured hexahedral intervals in the x direction, each divided into six conforming tetrahedra, for a total of 24 tetrahedra. The periodic physical-surface tags are

Direction	Minus tag	Plus tag
x	1	2
y	3	4
z	5	6

The corresponding plus-to-minus translations are $(-2, 0, 0)$, $(0, -1, 0)$, and $(0, 0, -1)$. These tags must map to `MaxwellBC_None` in the ordinary boundary registry because the periodic surface operator handles them separately.

The domain dimensions are not hard-coded requirements. For a conforming tetrahedral mesh of an axis-aligned box whose opposite surface triangulations match under translation, the driver infers

$$(x_{\min}, y_{\min}, z_{\min}), \qquad (x_{\max}, y_{\max}, z_{\max})$$

from the mesh coordinates. It reconstructs boundary tags 1 through 6 from the six coordinate planes and derives the periodic translations from $L_x = x_{\max} - x_{\min}$, $L_y = y_{\max} - y_{\min}$, and $L_z = z_{\max} - z_{\min}$. A separate domain-bounds command-line option is intentionally unnecessary because the mesh is the authoritative geometry description.

9.3.1 Generating meshes with a configurable x resolution

The generator `examples/meshes/generate_periodic_box_2x1x1_nx4.jl` accepts the number of structured cells in the x direction through `-nx`. Each cell is decomposed into six tetrahedra. For example:

```
julia --project=. \
  examples/meshes/generate_periodic_box_2x1x1.jl \
  --nx=8
```

This creates `examples/meshes/periodic_box_2x1x1_nx8.vtk`, with 36 points and 48 tetrahedra. The boundary tags remain 1/2, 3/4, and 5/6 for the x, y, and z periodic pairs. The general mesh sizes are

$$N_{\text{points}} = 4(n_x + 1), \qquad N_{\text{tet}} = 6n_x.$$

An explicit output path may be selected with `-output=PATH`. If `-nx` is omitted, the default value is four and the generated VTK file is identical to the supplied `nx4` mesh.

The analytical solution is

$$\boldsymbol{E}(\boldsymbol{x}, t) = \left(0, 0, -\frac{kH_0}{\omega\varepsilon_0}\right) \sin(kx - \omega t), \tag{9.1}$$

$$\boldsymbol{H}(\boldsymbol{x}, t) = (0, H_0, 0) \sin(kx - \omega t), \tag{9.2}$$

with

$$k = \omega = 2\pi, \qquad H_0 = 1, \qquad \varepsilon_0 = \mu_0 = 1.$$

Thus the implemented fields are

$$\boldsymbol{E} = (0, 0, -\sin(2\pi x - 2\pi t)), \qquad \boldsymbol{H} = (0, \sin(2\pi x - 2\pi t), 0).$$

On a translated box, the phase uses $x - x_{\min}$. The wave number can be selected with `-wave-number=K`; the angular frequency is $\omega = K/\sqrt{\varepsilon_0\mu_0}$. The driver requires

$$\frac{KL_x}{2\pi} \in \mathbb{N},$$

which ensures that the analytical field is periodic across the inferred x-boundaries. The default remains $K = 2\pi$.

Run two MPI partitions with:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --partitions=2
```

The value of `-partitions` must equal the number of MPI ranks. `-partitions=N` specifies a count, while `-partition=PATH` selects an existing METIS element-partition file. In particular, `-partition=2` means "read the file named `2`"; it does not request two partitions.

The generated `nx8` mesh can be run on two ranks with:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --partitions=2 \
  --mesh=examples/meshes/periodic_box_2x1x1_nx8.vtk \
  --repartition \
  --rebuild-distributed-mesh \
  --order=2 \
  --esprk-order=3 \
  --final-time=10.0 \
  --cfl=0.5
```

The default partition path and distributed-mesh cache contain the source mesh basename, so `nx4` and `nx8` artifacts do not collide. The `-repartition` and `-rebuild-distributed-mesh` options are required when replacing a mesh while retaining the same filename.

For example, an unstructured mesh with bounds $[0, 1] \times [0, 0.25] \times [0, 0.25]$ can be run directly:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --mesh=examples/meshes/periodic_box_unstructured.vtk \
  --partitions=2 \
  --repartition \
  --rebuild-distributed-mesh \
  --order=4 \
  --esprk-order=5 \
  --cfl=0.8 \
  --final-time=1.0
```

Here the default $K = 2\pi$ gives one wavelength across the unit x extent.

A four-way partition is generated and run with:

```
mpiexec -n 4 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --partitions=4 --repartition
```

9.3.2 Automatic METIS partitioning

When `-partition` is omitted, rank zero:

1. reads the legacy VTK tetrahedra;
2. calls `write_metis_mesh_from_vtk`;
3. runs the installed `mpmetis` executable for the requested number of partitions;
4. validates the zero-based element partition;
5. prepares the rank-local owned-plus-halo mesh cache.

Generated partitions are stored under `output/partitions/`. Use `-repartition` to replace an existing partition and `-rebuild-distributed-mesh` to recreate the rank-local cache. An existing partition may be selected explicitly:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --partitions=2 \
  --partition=output/partitions/periodic_box_2x1x1_nx4.mesh.epart.2
```

9.3.3 Polynomial-order requirement

This driver requires DG order $N \geq 2$. For $N = 1$ on the default `nx4` mesh, all DG nodes have $x \in \{0, 0.5, 1, 1.5, 2\}$, where $\sin(2\pi x) = 0$. Nodal interpolation would therefore produce an identically zero initial field even though the continuous analytical wave is nonzero inside each interval. The driver rejects `-order=1`.

9.3.4 Diagnostics and production output

The default output directory is `output/distributed_periodic_poisson_bracket/`. The driver preserves the production I/O behavior of the PEC Poisson-bracket case:

- `diagnostics/energy.csv`: DG mass-matrix electric, magnetic, and total energy;
- `diagnostics/quadrature_diagnostics.csv`: independent-cubature field and energy errors, optical chirality, electric and magnetic charge, and linear and angular momentum;
- `fields.pvd` and `fields/fields_stepNNNNNNNN.pvtu`: ParaView time series;
- `diagnostics/integration_points_rankNNNN.csv`: final integration-point numerical and analytical values;
- `config/run_metadata.toml`, `config/resolved_config.toml`, and `config/partition_metadata.csv`;
- `input/input_manifest.toml`;
- `checkpoints/stepNNNNNNNN/`: checksummed rank-local restart data;
- `status.toml`: current run status.

For backward compatibility, the run root also contains symlinks named `energy.csv`, `quadrature_diagnostics.csv`, `run_metadata.toml`, and `partition_metadata.csv`.

At every cubature point, the driver evaluates the optical chirality density

$$\chi = \frac{1}{2}\left(\varepsilon\, \boldsymbol{E} \cdot \nabla \times \boldsymbol{E} + \mu\, \boldsymbol{H} \cdot \nabla \times \boldsymbol{H}\right).$$

The curls are formed with the physical element derivative operators. The diagnostics file stores the distributed integral of χ, its analytical value and error, and the numerical, analytical, and error L^2 norms of its density. The rank-local integration-point files store the corresponding pointwise densities. For the linearly polarized traveling wave used here, the analytical optical chirality is identically zero.

Restart requires the same rank count, partition, and DG order:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --partitions=2 \
  --restart=output/distributed_periodic_poisson_bracket/checkpoints/step00000100 \
  --final-time=0.5
```

The implementation has been exercised with automatic two-way and four-way METIS partitions. In the short regression runs, the centered Poisson-bracket/ESPRK discretization preserved discrete total energy to roundoff and the integrated electric and magnetic charges remained near machine precision.

9.4 Distributed nonlinear PML pulse

The executable example `examples/distributed_nonlinear_pml_maxwell.jl` demonstrates the formulation from section 2.5. It generates $[0,2] \times [0,1] \times [0,1]$ in memory, partitions its $6n_x$ tetrahedra by x slab, uses periodic boundaries in y and z, and places polynomial nonlinear PML layers at both x ends. The initial condition is a right-traveling, linearly polarized Gaussian pulse:

$$\boldsymbol{E} = (0, 0, g(x)), \qquad \boldsymbol{H} = (0, -g(x), 0).$$

The default Hesthaven–Warburton upwind flux supplies a robust outer termination beyond the PML.

```
mpiexec -n 2 julia --project=. \
  examples/distributed_nonlinear_pml_maxwell.jl \
  --nx=8 --order=2 --rk-order=4 \
  --final-time=1.3 --cfl=0.15 \
  --pml-width=0.5 --sigma-max=12 \
  --sigma-degree=2
```

Important options are:

`-nx` Number of structured x cells; there are six tetrahedra per cell. It must be at least the MPI rank count.

`-pml-width`
Width of each x-directed layer. It must lie in $(0, 1)$ so the two layers do not overlap.

`-sigma-max` and `-sigma-degree`
Peak damping and polynomial profile degree.

`-regularization`
Positive δ added to both nonlinear denominators.

`-energy-every`
Number of time steps between global-energy samples.

Rank zero writes `output/distributed_nonlinear_pml/energy.csv`, containing the time, step, electric energy, magnetic energy, total energy, and total-to-initial energy ratio. A representative two-rank run with the command above reduced the ratio to approximately 1.25×10^{-2} by $t = 1.3$, with no increase between sampled energy values.

9.5 Empty-domain incident-wave PML gate

The first validation-sequence gate for the nonlinear PML is `examples/validate_empty_domain_incident_pml.j`. It is a thin release-gate wrapper around the production Poisson-bracket PML validation driver, so it exercises the same H-first ESPRK and nonlinear PML code path. The default case launches the same right-going sine-modulated Gaussian pulse in an otherwise empty cuboid, uses periodic boundaries in y and z, and places nonlinear PML layers at the two x boundaries. The default mesh is `examples/meshes/periodic_box_structured_nx16_ny8_nz8.vtk`; with $\lambda_0 = 0.5$ this gives four x-elements per central wavelength.

```
mpiexec -n 2 julia --project=. \
  examples/validate_empty_domain_incident_pml.jl \
  --max-reflection-ratio=1e-2 \
  --max-final-energy-ratio=2.5e-1
```

The pass/fail quantities are computed from the quadrature diagnostics written by the production PML driver:

`reflection_ratio`
: Maximum left-monitor energy after the configured reflection-start time, normalized by the initial total electromagnetic energy. This is the main empty-domain PML reflection metric.

`final_energy_ratio`
: Final total electromagnetic energy divided by the initial total energy. Since the PML is dissipative, the release gate requires this ratio to stay below `-max-final-energy-ratio`.

`max_energy_growth`
: Numerical tolerance for energy growth above the initial energy. The PML path should not generate energy, even though it is not energy-conservative.

The root output directory is `output/validation_sequence/empty_domain_incident_pml`. It contains the normal per-case PML run directories plus `empty_domain_incident_pml_summary.csv` and `empty_domain_incident_pml_summary.json`. The CSV and JSON files are machine-readable release-gate summaries. The executable exits with a nonzero status when any case fails. All options accepted by `distributed_poisson_bracket_pml_validation.jl` can be passed to this gate and override the defaults, including `-mesh`, `-flux`, `-pml-width`, `-sigma-max`, `-sigma-degree`, `-energy-every`, and `-paraview-every`.

9.6 PEC sphere boundary-residual gate

The second validation-sequence gate checks the incident-aware PEC condition on the metallic sphere. The numerical solution is stored as the scattered field, so the physical boundary condition is evaluated on the total field

$$\boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{E}_{\mathrm{scat}} + \boldsymbol{E}_{\mathrm{inc}}, \qquad \boldsymbol{n} \times \boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{0} \quad \text{on } \Gamma_{\mathrm{PEC}}.$$

The driver computes the surface diagnostic

$$\|\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{tot}}\|_{L^2(\Gamma_{\mathrm{PEC}})} = \left(\int_{\Gamma_{\mathrm{PEC}}} |\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{tot}}|^2 \, dS \right)^{1/2},$$

using the same owned sphere-face quadrature infrastructure used by the RCS surface-current diagnostic. It also reports the RMS residual, the RMS residual divided by the incident amplitude, and the maximum sampled pointwise residual.

```
mpiexec -n 2 julia --project=. \
  examples/validate_pec_sphere_boundary_residual.jl \
  --max-final-relative-rms=1e-1
```

The scattering run writes `diagnostics/pec_boundary_residual.csv`. The validation gate writes `pec_sphere_boundary_residual_summary.csv` and `pec_sphere_boundary_residual_summary.json` in `output/validation_sequence/pec_sphere_boundary_residual`. The primary pass/fail metric is `final_relative_rms`, controlled by `-max-final-relative-rms`. Additional optional gates are `-max-final-l2`, `-max-final-relative-max`, and `-max-sampled-relative-rms`. The

sampled maximum includes the startup transient unless the user selects diagnostics and final time so that startup samples are excluded from the gate. By default this gate uses `examples/meshes/metallic_sphere_scattering_validation.vtk`, a coarse validation mesh with roughly 2.9×10^4 tetrahedra. The larger `metallic_sphere_scattering.vtk` mesh has roughly 3.6×10^5 tetrahedra and should be reserved for production scattering/RCS studies.

9.7 Qualitative scattered E_x/H_y field gate

The next validation-sequence step is qualitative: it prepares the field output used to inspect the scattered electric and magnetic components from the metallic-sphere problem. The executable is `examples/validate_scattered_ex_hy_fields.jl`. It runs the scattering driver on the coarse validation sphere mesh, forces ParaView output, and checks that the final `.pvtu` snapshot contains the vector arrays `E_scat` and `H_scat`.

```
mpiexec -n 2 julia --project=. \
  examples/validate_scattered_ex_hy_fields.jl
```

The gate writes `fields.pvd` and the corresponding parallel VTK snapshots under `output/validation_sequence/scattered_ex_hy_fields`. Open `fields.pvd` in ParaView and display the components `E_scat_x` and `H_scat_y`. This step intentionally does not assign a strict numerical pass/fail criterion to the visual pattern. Its machine-readable summary, `scattered_ex_hy_fields_summary.json`, records the final PVD/PVTU paths, confirms the required scattered-field arrays are present, and checks that the final scattered energy is above `-min-final-scattered-energy`.

9.8 Coarse PEC-sphere RCS gate

The next validation-sequence step exercises the near-to-far RCS extraction on the coarse metallic-sphere mesh and compares the angular samples with the exact PEC Mie-series solution. The executable is `examples/validate_coarse_sphere_rcs.jl`. It is intentionally a coarse release gate: by default it checks that the RCS pipeline runs, that the accumulated Fourier samples are finite and nonzero, and that a machine-readable Mie comparison is written. The implementation details are documented in `docs/numerics/rcs_metallic_sphere.md`.

The production scattering driver is `examples/distributed_metallic_sphere_scattering.jl`. It solves for the scattered field of a PEC sphere illuminated by a plane wave propagating in the $+z$ direction with x-polarized electric field,

$$\boldsymbol{E}_{\mathrm{inc}} = A\hat{\boldsymbol{x}}\cos(kz - \omega t), \qquad \boldsymbol{H}_{\mathrm{inc}} = \frac{A}{Z}\hat{\boldsymbol{y}}\cos(kz - \omega t),$$

where $k = 2\pi/\lambda$, $\omega = ck$, $c = (\epsilon\mu)^{-1/2}$, and $Z = (\mu/\epsilon)^{1/2}$. The numerical unknown is the scattered field and the physical field is reconstructed as

$$\boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{E}_{\mathrm{scat}} + \boldsymbol{E}_{\mathrm{inc}}, \qquad \boldsymbol{H}_{\mathrm{tot}} = \boldsymbol{H}_{\mathrm{scat}} + \boldsymbol{H}_{\mathrm{inc}}.$$

The incident-aware PEC boundary condition on the sphere is

$$\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{0}, \qquad \boldsymbol{n} \times \boldsymbol{E}_{\mathrm{scat}} = -\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{inc}}.$$

When RCS extraction is enabled, the driver forms the PEC equivalent surface current

$$\boldsymbol{J}_s = \boldsymbol{n} \times \boldsymbol{H}_{\mathrm{tot}}$$

and accumulates the near-to-far integral

$$\boldsymbol{I}(\hat{\boldsymbol{r}}, t) = \int_{\Gamma_{\mathrm{PEC}}} \boldsymbol{J}_s(\boldsymbol{x}, t) \exp(-ik\, \hat{\boldsymbol{r}} \cdot \boldsymbol{x})\; dS.$$

The time-domain integral is converted to a phasor through a Hann-windowed Fourier extraction over [`-rcs-start-time`, `-final-time`]. The reconstructed transverse far field is then normalized as

$$\sigma(\theta, \phi) = 4\pi \frac{|\boldsymbol{E}^{\infty}_{\mathrm{scat}}(\theta, \phi)|^2}{|\boldsymbol{E}_{\mathrm{inc}}|^2}.$$

```
mpiexec -n 2 julia --project=. \
  examples/validate_coarse_sphere_rcs.jl
```

The default run uses `examples/meshes/metallic_sphere_scattering_validation.vtk`, DG order $p = 1$, ESPRK order 2, `-enable-rcs`, a 37-point polar sampling grid at $\phi = 0$, and disables ParaView snapshots. The scattering driver writes the numerical far-field table `diagnostics/rcs.csv`. The validation gate writes `diagnostics/coarse_mie_rcs_comparison.csv`, with one row per observation direction and columns for

$$\sigma_h(\theta, \phi), \qquad \sigma_{\mathrm{Mie}}(\theta, \phi), \qquad |\sigma_h - \sigma_{\mathrm{Mie}}|, \qquad \frac{|\sigma_h - \sigma_{\mathrm{Mie}}|}{\pi a^2},$$

together with relative and dB errors. It also writes `coarse_sphere_rcs_summary.csv` and `coarse_sphere_rcs_summary.json` in `output/validation_sequence/coarse_sphere_rcs`.

The default pass/fail criteria are deliberately loose because a short coarse run is not a resolved RCS benchmark. The gate requires finite RCS rows, at least one accumulated sample, positive window weight, and `max_numerical_rcs >= -min-max-rcs`. The default `-min-max-rcs` is 10^{-12}. Stricter quantitative checks can be enabled with `-max-absolute-normalized-error`, `-max-rms-relative-error`, and `-max-db-error`. Those thresholds should be selected only after choosing a resolved mesh, final time, and Fourier-window start time.

The RCS command-line controls have the following meaning:

`-rcs-start-time`
: First time included in the Fourier extraction. For quantitative Mie comparison, this should be after the startup transient has left the near field. Setting it equal to the final time is useful only for one-step smoke tests.

`-rcs-every`
: Time-step interval between RCS samples. Small values increase sampling but also increase surface-integration overhead. Long production runs usually do not require `-rcs-every=1`.

`-rcs-theta-count`, `-rcs-theta-min-degrees`, `-rcs-theta-max-degrees`
: Polar scattering-angle grid. The default coarse gate uses 37 samples over $[0°, 180°]$.

`-rcs-phi-degrees`
: Comma-separated azimuth angles. The default $\phi = 0°$ is the E-plane cut for the current incident polarization.

For a lighter high-order smoke check, generate the four-elements-per-wavelength mesh near the sphere and run the same gate with $p = 4$ and ESPRK5:

```
gmsh examples/meshes/metallic_sphere_scattering.geo -3 -format vtk \
  -o examples/meshes/metallic_sphere_scattering_epw4.vtk \
  -setnumber ElementsPerWavelength 4 \
  -setnumber SphereMeshSizeFactor 1.0 \
  -setnumber PMLMeshSizeFactor 3.0 \
```

```
  -setnumber FarMeshSizeFactor 4.0 \
  -setnumber CurvatureSamples 8 \
  -setnumber PMLWidth 1.0 \
  -setnumber AirBuffer 1.0 \
  -setnumber OptimizeMesh 0

mpiexec -n 2 -genv OPENBLAS_NUM_THREADS 1 julia --project=. \
  examples/validate_coarse_sphere_rcs.jl \
  --mesh=examples/meshes/metallic_sphere_scattering_epw4.vtk \
  --order=4 --esprk-order=5 \
  --final-time=1e-6 --rcs-start-time=1e-6 \
  --rcs-every=1 --diagnostics-every=1 \
  --min-max-rcs=0 \
  --output-dir=output/validation_sequence/coarse_sphere_rcs_epw4_p4_smoke
```

The resulting `diagnostics/rcs.csv` can be processed with the existing Mie comparison script:

```
python3 examples/postprocess_metallic_sphere_mie_rcs.py \
  --run-dir output/validation_sequence/coarse_sphere_rcs_epw4_p4_smoke
```

The script writes `diagnostics/mie_rcs_comparison.csv`, `tables/mie_rcs_comparison.tex`, and `plots/mie_rcs_comparison.pdf`. A one-step high-order smoke run validates the setup, RCS extraction, and postprocessing path; it is not a quantitative RCS benchmark. For Mie agreement, use a longer final time and start the Fourier extraction after the transient has left the near field. The postprocessor figure has two panels: the top panel compares $10\log_{10}(\sigma/(\pi a^2))$ with the exact Mie curve, while the lower panel shows $|\sigma_h - \sigma_{\mathrm{Mie}}|/|\sigma_{\mathrm{Mie}}|$. Large errors in a one-step or startup-window run indicate that the Fourier estimate is not a steady monochromatic RCS; they do not by themselves prove that the output pipeline is wrong. A resolved validation should be stable when `-rcs-start-time`, `-rcs-every`, the mesh, and the polynomial order are varied.

9.9 Production Poisson-bracket PML validation

The production PML validation executable is `examples/distributed_poisson_bracket_pml_validation.jl`. It uses the Poisson-bracket Maxwell operator, the H-first ESPRK integrator, periodic boundary pairs in y and z, and nonlinear PML layers at the two x boundaries. By default it reads `examples/meshes/periodic_box_structured_nx4_ny2_nz2.vtk`; pass `-mesh` for one mesh or `-meshes` for a comma-separated mesh sweep. Meshes must be axis-aligned cuboids; boundary IDs are inferred geometrically as 1/2 on $x_{\min}/x_{\max}$, 3/4 on $y_{\min}/y_{\max}$, and 5/6 on $z_{\min}/z_{\max}$. The loaded mesh is partitioned with a deterministic balanced spatial partition. The initial condition is a right-going sine-modulated Gaussian pulse,

$$g(x) = \exp\left[-\left(\frac{x-x_0}{w}\right)^2\right]\sin\left(k_0(x-x_0)\right), \qquad \boldsymbol{E} = (0,0,\sqrt{\mu/\epsilon}\,g(x)), \qquad \boldsymbol{H} = (0,-g(x),0).$$

By default $\lambda_0 = 0.5$, so $k_0 = 4\pi$ and, for $\epsilon = \mu = 1$, $f_0 = 2$. On `examples/meshes/periodic_box_structured_n` this is four x-elements per central wavelength.

```
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_pml_validation.jl \
  --mesh=examples/meshes/periodic_box_structured_nx16_ny8_nz8.vtk \
  --order=2 --esprk-order=3 \
  --final-time=3.0 --cfl=0.15 \
  --central-wavelength=0.5 \
  --fluxes=centered,alternating \
```

```
  --pml-widths=0.25,0.5 \
  --sigma-maxes=8,12 --sigma-degrees=2,3 \
  --paraview-every=25
```

The command above runs a Cartesian product sweep over mesh, flux kind, PML width, maximum damping, and damping polynomial degree. Each case gets its own isolated run directory below `output/poisson_bracket_pml_validation`. The root directory also contains `sweep_summary.csv`, with one row per case and columns for the mesh path, central frequency, central wavelength, central wavenumber, final energy ratio, reflection ratio, maximum reflected left-monitor energy, time step, step count, and case output directory.

The case diagnostics are:

`diagnostics/energy.csv`
: Global electric, magnetic, total, and component Hamiltonian energies.

`diagnostics/reflection_diagnostics.csv`
: Quadrature-integrated total energy, left-PML energy, right-PML energy, interior energy, left-monitor energy, right-monitor energy, the maximum left-monitor energy after the configured reflection start time, and the normalized reflection ratio.

`fields.pvd` and `fields/`
: ParaView time-series output using parallel high-order VTK Lagrange tetrahedra. The analytical arrays are zero placeholders for this validation pulse; the reflection diagnostics are the quantitative validation output.

`config/` and `input/`
: Resolved configuration, input manifest, run metadata, partition metadata, and run-layout metadata.

The default reflection start time is estimated from the pulse center, wave speed $c = 1/\sqrt{\epsilon\mu}$, and the right PML interface. Override it with `-reflection-start` when the monitor placement or pulse shape changes. The left monitor is `x <= -left-monitor-max`; the right monitor is `x >= -right-monitor-min`. Alternating Poisson-bracket fluxes currently support only PEC outer boundaries, so the validation driver rejects `-flux=alternating -outer-boundary=pmc` and `-flux=alternating -outer-boundary=absorbing` before time stepping.

9.10 Stationary PEC cube

```
julia --project=. examples/solver/stationary_pec_cube.jl \
  --cells=3 --order=2 --nsteps=20 --rk=4 --cfl=0.05 \
  --output=output/stationary_hw
```

The case uses the cube $[-1, 1]^3$, PEC walls with boundary tag 10, and a cavity mode with $\omega = \sqrt{3}\pi$. It prints energy, right-hand-side, and tangential-electric-field diagnostics and writes final VTU data.

Display its complete options with:

```
julia --project=. examples/solver/stationary_pec_cube.jl --help
```

9.11 Stationary central-flux ESPRK case

```
julia --project=. examples/solver/stationary_pec_cube_esprk.jl \
  --cells=3 --order=2 --nsteps=20 --esprk=4 --first=H \
  --cfl=0.05 --output=output/stationary_esprk
```

This case combines the Hesthaven–Warburton central flux with an explicit partitioned symplectic RK method. Both electric-first and magnetic-first stage orders are accepted.

9.12 Stationary Poisson-bracket ESPRK case

```
julia --project=. --threads=8 \
  examples/solver/stationary_pec_cube_pb_esprk.jl \
  --cells=3 --order=2 --nsteps=20 --esprk=4 \
  --backend=threaded --cfl=0.05 \
  --output=output/stationary_pb
```

The Poisson-bracket case requires `first=H`. It supports both the serial and threaded backend options.

9.13 Upwind convergence study

```
julia --project=. examples/solver/convergence_hw_upwind.jl \
  --cells=2,3,4,5 \
  --orders=1,2,4 \
  --time=0.005 \
  --cfl=0.1 \
  --rk=5 \
  --backend=serial \
  --output=output/convergence_hw_upwind.csv
```

For a threaded run:

```
julia --project=. --threads=8 \
  examples/solver/convergence_hw_upwind.jl \
  --cells=2,3,4,5 --orders=1,2,4 --backend=threaded
```

The driver requires exactly four mesh levels. It creates deterministic jittered tetrahedral cube meshes, computes continuous error norms, estimates convergence rates, and writes CSV output.

9.14 Central-flux partitioned convergence study

```
julia --project=. \
  examples/solver/convergence_hw_central_psrk_hfirst.jl \
  --cells=2,3,4,5 \
  --orders=1,2,4 \
  --psrk=2 \
  --first=H \
  --output=output/convergence_hw_central_psrk_hfirst.csv
```

9.15 Poisson-bracket threaded convergence study

```
julia --project=. --threads=8 \
  examples/solver/convergence_pb_esprk_threaded.jl \
  --cells=3,4,5,6 \
  --orders=1,2,3,4 \
  --seconds=10 \
  --backend=threaded \
  --output=output/convergence_pb_esprk_threaded.csv
```

For this validation, the ESPRK order is fixed to spatial order plus one. A second CSV file with `_3dec` inserted before the extension is also written.

9.16 Distributed Poisson-bracket convergence study

The MPI convergence driver uses PEC boundaries by default:

```
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_poisson_bracket_maxwell.jl \
  --cells=2,3,4,5 --orders=1,2,3
```

This default path uses in-memory structured unit-cube meshes. Each cube cell is split into six tetrahedra, and interior grid nodes are jittered by `-jitter` times the structured mesh spacing. Boundary nodes remain on the exact cube faces. The run uses a deterministic seed selected by `-seed`.

Select PMC boundaries and the electromagnetic-dual PMC cavity eigenmode with:

```
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc \
  --cells=2,3,4,5 --orders=1,2,3 \
  --output=output/convergence_distributed_poisson_bracket_pmc.csv
```

The accepted values are `pec` and `pmc`, case-insensitively, with `pec` as the default. The output CSV records the selection in its `boundary_condition` column.

The same driver can generate an unstructured Gmsh mesh family on the unit cube:

```
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_poisson_bracket_maxwell.jl \
  --mesh-family=unstructured \
  --nx-targets=2,4,8,16 \
  --orders=2,3,4 \
  --periods=1 \
  --output=output/convergence_distributed_poisson_bracket_unstructured.csv
```

Rank zero invokes Gmsh on `examples/meshes/unit_cube_unstructured.geo`, writes VTK meshes under `-mesh-dir`, and then uses the same root-distribution and spatial partitioning path as the structured study. If `-mesh-dir` is omitted, the directory is named `convergence_unstructured_meshes` beside the output CSV directory. The convenience options are:

- `-mesh-family=structured|unstructured`;
- `-nx-target=N -levels=L`, which generates $N, 2N, 4N, \ldots$;
- `-nx-targets=a,b,c`, which selects an explicit unstructured family and overrides the base target and level count;

- `-geo=PATH`, to select another parameterized Gmsh geometry;
- `-mesh-dir=PATH`, to preserve generated VTK files in a chosen directory.

For unstructured runs, the values printed as levels are Gmsh `NxTarget` values. They request $h_{\text{target}} = L_x/N_x$, but the rate calculation uses the measured volume-based

$$h_{\text{char}} = \left(\frac{|\Omega|}{N_{\text{el}}}\right)^{1/3}.$$

The CSV records `mesh_family` in addition to `boundary_condition`, so structured and unstructured cavity results are distinguishable in post-processing. The post-processing script also accepts older PEC CSV files without these newer columns.

9.17 Distributed periodic plane-wave convergence study

The periodic convergence driver uses the traveling plane wave from the distributed periodic experiment:

$$E = (0, 0, -\sin(2\pi x - 2\pi t)), \qquad H = (0, \sin(2\pi x - 2\pi t), 0).$$

Run one complete wave period with:

```
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_periodic_poisson_bracket_maxwell.jl \
  --nx-targets=4,8,16 \
  --orders=2,3,4 \
  --periods=1 \
  --output=output/convergence_distributed_periodic_poisson_bracket.csv
```

Rank zero invokes Gmsh on `examples/meshes/periodic_box_unstructured.geo`. The default `-nx-target=2 -levels=4` generates

$$N_x = 2, 4, 8, 16, \qquad h_\ell = \frac{L_x}{N_x} = h_0\, 2^{-\ell}, \quad \ell = 0, 1, 2, 3.$$

Equivalently, `-nx-targets=4,8,16` selects an explicit family and is a better starting point for production convergence checks on the current unstructured geometry. The result is an unstructured periodic tetrahedral mesh family with uniformly halved requested mesh size. Boundary tags $1 \leftrightarrow 2$, $3 \leftrightarrow 4$, and $5 \leftrightarrow 6$ are paired by the production distributed periodic exchange.

The driver records two different mesh-size measures:

$$h_{\text{target}} = \frac{L_x}{N_x}, \qquad h_{\text{char}} = \left(\frac{|\Omega|}{N_{\text{el}}}\right)^{1/3}.$$

The first quantity is stored as `target_h` and is the Gmsh request. The second quantity is stored as `characteristic_h` and is measured from the generated distributed mesh. Observed rates are computed from `characteristic_h`:

$$p = \frac{\log(e_{\text{coarse}}/e_{\text{fine}})}{\log(h_{\text{char,coarse}}/h_{\text{char,fine}})}.$$

This is deliberate. Independently generated unstructured Gmsh meshes are not nested, and doubling `NxTarget` does not guarantee that the realized tetrahedral volume scale is exactly halved. The console therefore also prints the ratios of `target_h` and `characteristic_h`. CSV files generated before this convention should be regenerated before rate comparisons.

The CSV schema matches the cavity convergence driver and records `periodic` in the `boundary_condition` column. It additionally records L^2 errors and rates for every component $E_x, E_y, E_z, H_x, H_y, H_z$, aggregate electric, magnetic, and total errors, `h_min`, `h_max`, energy error, charge diagnostics, and sampled L^∞ errors. For this analytical wave, the active components are E_z and H_y. The exact components E_x, E_y, H_x, H_z are zero, so their norms measure transverse polarization leakage. The console reports the combined leakage measures

$$\left(\|E_x\|_{L^2}^2 + \|E_y\|_{L^2}^2\right)^{1/2}, \qquad \left(\|H_x\|_{L^2}^2 + \|H_z\|_{L^2}^2\right)^{1/2}.$$

The finest refinement pair is checked against strict aggregate targets: electric rate $N+1$ and magnetic rate N, with the same half-order acceptance tolerance as the cavity study. This is a regression target, not a generic theorem for every unstructured tetrahedral family. On general unstructured meshes the exact-zero components can converge closer to order N; if those leakage components dominate the vector norm, the aggregate electric rate can fail the strict $N+1$ target even when the active E_z and H_y plane-wave components converge at the expected rate. Inspect the component table, leakage table, and mesh-size-ratio table before interpreting a failed strict verdict.

Chapter 10

Radar Cross Section: Theory, Conventions, and Numerical Evaluation

This document gives a self-contained description of radar cross section (RCS), the time-harmonic scattering model used for perfectly electrically conducting targets, and the numerical procedure implemented in DiscoGMPI. It fixes the physical assumptions, phase convention, angular convention, normalization, near-to-far transformation, time-windowed Fourier extraction, DG surface quadrature, MPI reduction, output files, and validation strategy. The focus is the metallic-sphere workflow implemented in `examples/distributed_metallic_sphere_scattering.jl`, with validation and Mie-series comparison through `examples/validate_coarse_sphere_rcs.jl` and `examples/postprocess_metallic_sphere_mie_rcs.py`. The document also describes the NASA/Austin PEC Almond benchmark workflow implemented in `examples/distributed_pec_almond_scattering` and `examples/postprocess_pec_almond_rcs.py`.

10.1 Purpose of RCS

Radar cross section is a far-field measure of how strongly a target scatters an incident electromagnetic wave into a selected observation direction. It is defined so that a distant receiver can interpret the scattered field as if it had been produced by an equivalent isotropic scatterer with area σ. Large RCS means that the object returns a strong scattered field in that direction; small RCS means that it scatters weakly.

For a monochromatic incident plane wave with electric-field amplitude $|\boldsymbol{E}_{\mathrm{inc}}|$, the far scattered field has the asymptotic form

$$\widehat{\boldsymbol{E}_{\mathrm{scat}}}(\boldsymbol{x}) = \frac{\mathrm{e}^{\mathrm{i}kr}}{r}\boldsymbol{E}^{\infty}(\widehat{\boldsymbol{r}}) + \mathcal{O}(r^{-2}), \qquad r = |\boldsymbol{x}|, \qquad \widehat{\boldsymbol{r}} = \frac{\boldsymbol{x}}{|\boldsymbol{x}|}. \tag{10.1}$$

The RCS in direction $\widehat{\boldsymbol{r}}$ is

$$\sigma(\widehat{\boldsymbol{r}}) = \lim_{r\to\infty} 4\pi r^2 \frac{|\widehat{\boldsymbol{E}_{\mathrm{scat}}}(r\widehat{\boldsymbol{r}})|^2}{|\widehat{\boldsymbol{E}_{\mathrm{inc}}}|^2} = 4\pi \frac{|\boldsymbol{E}^{\infty}(\widehat{\boldsymbol{r}})|^2}{|\widehat{\boldsymbol{E}_{\mathrm{inc}}}|^2}. \tag{10.2}$$

In DiscoGMPI, the incident amplitude is denoted by A, so $|\widehat{\boldsymbol{E}_{\mathrm{inc}}}| = A$. For a sphere of radius a, postprocessing usually plots the dimensionless normalized RCS

$$\frac{\sigma}{\pi a^2} \tag{10.3}$$

or its decibel value

$$10\log_{10}\left(\frac{\sigma}{\pi a^2}\right). \tag{10.4}$$

10.2 Physical Assumptions

The current DiscoGMPI metallic-sphere RCS workflow assumes:

1. The exterior medium is homogeneous, isotropic, and lossless outside the artificial absorbing layer.
2. The material parameters are scalar constants $\epsilon > 0$ and $\mu > 0$, with wave speed
$$c = \frac{1}{\sqrt{\epsilon\mu}}, \qquad Z = \sqrt{\frac{\mu}{\epsilon}},$$
where Z is the wave impedance.
3. The scatterer is a perfectly electrically conducting (PEC) sphere centered in a Cartesian computational box.
4. The numerical unknown is the scattered field. The physical field is reconstructed as incident plus scattered field.
5. The outer computational box is truncated with the nonlinear Cartesian PML available in DiscoGMPI.
6. The RCS diagnostic is extracted from the PEC surface current. The current implementation does not yet use a Huygens box or volume near-to-far transformation.
7. The incident wave is monochromatic and the reported RCS is the Fourier component at the incident frequency. Time-domain transients must be excluded by choosing an appropriate `-rcs-start-time`.

10.3 Time-Harmonic Convention

DiscoGMPI uses the real incident wave

$$\boldsymbol{E}_{\mathrm{inc}}(\boldsymbol{x}, t) = A\,\widehat{\boldsymbol{x}}\cos(kz - \omega t) \tag{10.5}$$

for the default sphere experiment. This is equivalent to the phasor convention

$$\boldsymbol{E}(\boldsymbol{x}, t) = \mathrm{Re}\left\{\widehat{\boldsymbol{E}}(\boldsymbol{x})\,\mathrm{e}^{-\mathrm{i}\omega t}\right\}. \tag{10.6}$$

With this convention, the default incident phasors are

$$\widehat{\boldsymbol{E}_{\mathrm{inc}}}(\boldsymbol{x}) = A\,\widehat{\boldsymbol{x}}\,\mathrm{e}^{\mathrm{i}kz}, \qquad \widehat{\boldsymbol{H}_{\mathrm{inc}}}(\boldsymbol{x}) = \frac{A}{Z}\,\widehat{\boldsymbol{y}}\,\mathrm{e}^{\mathrm{i}kz}. \tag{10.7}$$

The corresponding real magnetic field is

$$\boldsymbol{H}_{\mathrm{inc}}(\boldsymbol{x}, t) = \frac{A}{Z}\,\widehat{\boldsymbol{y}}\cos(kz - \omega t). \tag{10.8}$$

The general plane-wave form is

$$\widehat{\boldsymbol{E}_{\mathrm{inc}}}(\boldsymbol{x}) = A\,\boldsymbol{p}\,\mathrm{e}^{\mathrm{i}k\boldsymbol{d}\cdot\boldsymbol{x}}, \qquad \widehat{\boldsymbol{H}_{\mathrm{inc}}}(\boldsymbol{x}) = \frac{A}{Z}\,(\boldsymbol{d}\times\boldsymbol{p})\,\mathrm{e}^{\mathrm{i}k\boldsymbol{d}\cdot\boldsymbol{x}}, \tag{10.9}$$

where $\boldsymbol{d}$ is the unit propagation direction, $\boldsymbol{p}$ is the unit electric polarization, and $\boldsymbol{d}\cdot\boldsymbol{p} = 0$. The default code values are

$$\boldsymbol{d} = \widehat{\boldsymbol{z}}, \qquad \boldsymbol{p} = \widehat{\boldsymbol{x}}.$$

10.4 Maxwell Scattering Problem

In the physical exterior domain Ω^+, the total fields satisfy

$$\epsilon \frac{\partial \boldsymbol{E}_{\mathrm{tot}}}{\partial t} = \nabla \times \boldsymbol{H}_{\mathrm{tot}}, \tag{10.10}$$

$$\mu \frac{\partial \boldsymbol{H}_{\mathrm{tot}}}{\partial t} = -\nabla \times \boldsymbol{E}_{\mathrm{tot}}. \tag{10.11}$$

In phasor form, using $\mathrm{e}^{-\mathrm{i}\omega t}$,

$$-\mathrm{i}\omega\epsilon\, \widehat{\boldsymbol{E}_{\mathrm{tot}}} = \nabla \times \widehat{\boldsymbol{H}_{\mathrm{tot}}}, \tag{10.12}$$

$$-\mathrm{i}\omega\mu\, \widehat{\boldsymbol{H}_{\mathrm{tot}}} = -\nabla \times \widehat{\boldsymbol{E}_{\mathrm{tot}}}. \tag{10.13}$$

The wavenumber and angular frequency are related by

$$k = \frac{\omega}{c} = \omega\sqrt{\epsilon\mu} = \frac{2\pi}{\lambda}. \tag{10.14}$$

The total field is decomposed as

$$\boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{E}_{\mathrm{inc}} + \boldsymbol{E}_{\mathrm{scat}}, \qquad \boldsymbol{H}_{\mathrm{tot}} = \boldsymbol{H}_{\mathrm{inc}} + \boldsymbol{H}_{\mathrm{scat}}. \tag{10.15}$$

The numerical field stored by the driver is

$$\boldsymbol{U}_h = \left(\boldsymbol{E}_{\mathrm{scat},h}, \boldsymbol{H}_{\mathrm{scat},h}\right). \tag{10.16}$$

This is useful because the incident field is known analytically everywhere, including on the metallic surface and in the PML region.

10.4.1 PEC Boundary Condition

On the metallic sphere Γ_{PEC}, the total tangential electric field must vanish:

$$\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{tot}} = \boldsymbol{0} \qquad \text{on } \Gamma_{\mathrm{PEC}}. \tag{10.17}$$

Using the scattered-field decomposition, the condition imposed by the code is

$$\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{scat}} = -\boldsymbol{n} \times \boldsymbol{E}_{\mathrm{inc}} \qquad \text{on } \Gamma_{\mathrm{PEC}}. \tag{10.18}$$

Here $\boldsymbol{n}$ is the normal pointing out of the PEC object into the exterior medium. The DG mesh is a box with an inner spherical hole; the geometric boundary normal stored for the exterior computational domain points opposite to the PEC-object normal on the inner sphere. Therefore `build_pec_sphere_face_samples` flips the boundary-face normal before using it in the PEC residual and RCS current:

$$\boldsymbol{n}_{\mathrm{RCS}} = -\boldsymbol{n}_{\mathrm{DG,inner}}. \tag{10.19}$$

10.4.2 Absorbing Layer

The physical scattering problem is posed in an unbounded exterior domain. The production driver truncates the domain with a Cartesian nonlinear PML near the outer box boundaries. The RCS diagnostic is not evaluated on the outer PML; it is evaluated on the PEC sphere surface. A reliable RCS run therefore requires the PML to suppress outgoing scattered waves without reflecting them back to the sphere during the Fourier accumulation window.

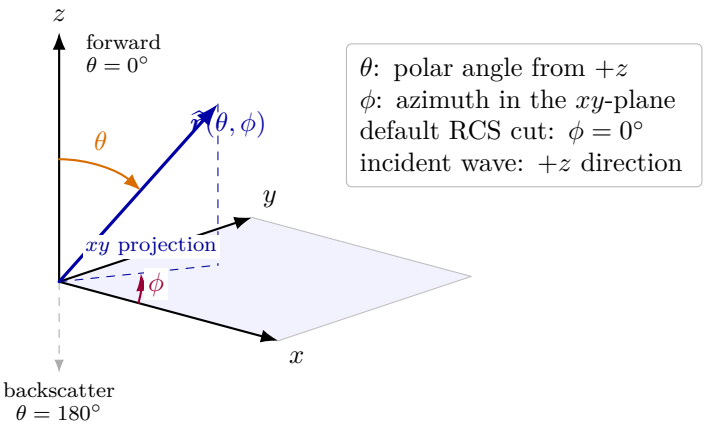


Figure 10.1: DiscoGMPI angular convention for RCS observation directions. The polar angle θ is measured from the $+z$ axis, while the azimuth ϕ is measured in the xy-plane from $+x$ toward $+y$.

10.5 Angular Coordinates

DiscoGMPI uses the standard spherical convention

$$\hat{\boldsymbol{r}}(\theta,\phi) = (\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta)\,, \tag{10.20}$$

with

$$0 \le \theta \le \pi, \qquad 0 \le \phi < 2\pi.$$

Here θ is the polar scattering angle measured away from the positive z-axis, and ϕ is the azimuthal angle in the xy-plane measured from the positive x-axis toward the positive y-axis. The convention is shown in Figure 10.1. For the default incident wave propagating in $+z$, $\theta = 0°$ is forward scattering and $\theta = 180°$ is backscatter.

The transverse basis vectors are

$$\hat{\boldsymbol{e}}_\theta = (\cos\theta\cos\phi,\ \cos\theta\sin\phi,\ -\sin\theta)\,, \tag{10.21}$$

$$\hat{\boldsymbol{e}}_\phi = (-\sin\phi,\ \cos\phi,\ 0)\,. \tag{10.22}$$

The reported far-field components are

$$E_\theta^\infty = \hat{\boldsymbol{e}}_\theta \cdot \boldsymbol{E}^\infty, \qquad E_\phi^\infty = \hat{\boldsymbol{e}}_\phi \cdot \boldsymbol{E}^\infty. \tag{10.23}$$

The RCS uses the transverse magnitude

$$|\boldsymbol{E}^\infty|^2 = |E_\theta^\infty|^2 + |E_\phi^\infty|^2. \tag{10.24}$$

For the default $+z$-propagating incident wave, $\theta = 0°$ is the forward direction and $\theta = 180°$ is the backscatter direction. The default RCS comparison uses the E-plane cut

$$\phi = 0°, \qquad 0° \le \theta \le 180°.$$

Some papers use a different naming convention for the angular variables. If a paper says "$\theta = 0$, $0 \le \phi \le 180°$" for a plot that varies from forward to backscatter, its ϕ is the physical scattering angle. In DiscoGMPI, the same physical plot is produced with

$$\phi = 0°, \qquad \theta \in [0°, 180°].$$

At $\theta = 0°$ and $\theta = 180°$, the azimuthal basis is geometrically degenerate: all values of ϕ describe the same Cartesian observation direction, but the transverse basis rotates. A $\theta = 0°$, $0 \le \phi \le 180°$ plot is therefore a basis orientation check, not the usual forward-to-backward RCS cut.

10.6 Near-To-Far Transformation

10.6.1 Outgoing Green Phase

With the phasor convention $\mathrm{e}^{-\mathrm{i}\omega t}$, the outgoing scalar Green function has the phase

$$G(\boldsymbol{x},\boldsymbol{y}) = \frac{\mathrm{e}^{\mathrm{i}k|\boldsymbol{x}-\boldsymbol{y}|}}{4\pi|\boldsymbol{x}-\boldsymbol{y}|}. \tag{10.25}$$

For a far-field observation point $\boldsymbol{x} = r\widehat{\boldsymbol{r}}$,

$$|\boldsymbol{x}-\boldsymbol{y}| = r - \widehat{\boldsymbol{r}}\cdot\boldsymbol{y} + \mathcal{O}(r^{-1}). \tag{10.26}$$

Thus

$$\mathrm{e}^{\mathrm{i}k|\boldsymbol{x}-\boldsymbol{y}|} = \mathrm{e}^{\mathrm{i}kr}\mathrm{e}^{-\mathrm{i}k\widehat{\boldsymbol{r}}\cdot\boldsymbol{y}}\left(1+\mathcal{O}(r^{-1})\right). \tag{10.27}$$

This is why the DiscoGMPI near-to-far surface integral uses

$$\exp(-\mathrm{i}k\,\widehat{\boldsymbol{r}}\cdot\boldsymbol{y}) \tag{10.28}$$

for the spatial phase factor. Using the opposite sign would evaluate the same Fourier transform in direction $-\widehat{\boldsymbol{r}}$, effectively reflecting an E-plane RCS curve by the mapping

$$\theta \mapsto 180^\circ - \theta.$$

10.6.2 PEC Equivalent Surface Current

For a PEC scatterer, the relevant equivalent electric surface current is

$$\widehat{\boldsymbol{J}}_s = \boldsymbol{n}\times\widehat{\boldsymbol{H}_{\mathrm{tot}}} \qquad \text{on } \Gamma_{\mathrm{PEC}}. \tag{10.29}$$

The code evaluates this using the total magnetic field

$$\boldsymbol{H}_{\mathrm{tot},h}(\boldsymbol{x},t) = \boldsymbol{H}_{\mathrm{scat},h}(\boldsymbol{x},t) + \boldsymbol{H}_{\mathrm{inc}}(\boldsymbol{x},t). \tag{10.30}$$

The time-dependent surface-current integral for an observation direction $\widehat{\boldsymbol{r}}$ is

$$\boldsymbol{I}(\widehat{\boldsymbol{r}},t) = \int_{\Gamma_{\mathrm{PEC}}} \boldsymbol{J}_s(\boldsymbol{y},t)\mathrm{e}^{-\mathrm{i}k\widehat{\boldsymbol{r}}\cdot\boldsymbol{y}}\,\mathrm{d}S_y. \tag{10.31}$$

After Fourier extraction, the phasor integral is denoted $\widehat{\boldsymbol{I}}(\widehat{\boldsymbol{r}})$.

10.6.3 Far-Field Reconstruction

The far-field amplitude used in DiscoGMPI is

$$\boldsymbol{E}^{\infty}(\widehat{\boldsymbol{r}}) = \frac{\mathrm{i}kZ}{4\pi}\left[\widehat{\boldsymbol{r}}\left(\widehat{\boldsymbol{r}}\cdot\widehat{\boldsymbol{I}}(\widehat{\boldsymbol{r}})\right) - \widehat{\boldsymbol{I}}(\widehat{\boldsymbol{r}})\right]. \tag{10.32}$$

Equivalently,

$$\boldsymbol{E}^{\infty}(\widehat{\boldsymbol{r}}) = \frac{\mathrm{i}kZ}{4\pi}\widehat{\boldsymbol{r}}\times\left(\widehat{\boldsymbol{r}}\times\widehat{\boldsymbol{I}}(\widehat{\boldsymbol{r}})\right). \tag{10.33}$$

The vector in brackets is transverse because

$$\widehat{\boldsymbol{r}}\cdot\boldsymbol{E}^{\infty} = 0.$$

The overall sign convention depends on the normal convention and equivalent current convention. The RCS depends on $|\boldsymbol{E}^{\infty}|^2$, so a global sign does not affect the reported RCS.

Combining (10.2) and (10.32), the code reports

$$\sigma_h(\theta,\phi) = 4\pi\frac{|E^{\infty}_{\theta,h}|^2 + |E^{\infty}_{\phi,h}|^2}{A^2}. \tag{10.34}$$

10.7 Time-Domain Fourier Extraction

The solver advances real-valued fields in time. To obtain a monochromatic RCS, DiscoGMPI extracts the phasor at the incident frequency. If

$$f(t) = \mathrm{Re}\{\widehat{f}\,\mathrm{e}^{-\mathrm{i}\omega t}\},$$

then, over an integer number of periods,

$$\widehat{f} = \frac{2}{T}\int_0^T f(t)\,\mathrm{e}^{\mathrm{i}\omega t}\,\mathrm{d}t.$$

DiscoGMPI uses the discrete Hann-windowed estimator

$$\widehat{\boldsymbol{I}}_h(\widehat{\boldsymbol{r}}) = \frac{2}{W}\sum_m w_m\,\boldsymbol{I}_h(\widehat{\boldsymbol{r}}, t_m)\mathrm{e}^{\mathrm{i}\omega t_m}, \qquad W = \sum_m w_m. \tag{10.35}$$

The samples t_m are selected every `-rcs-every` time steps, and the final step is included. The current production drivers use a fixed adjusted time step, so the constant time-step factor cancels between numerator and denominator in (10.35).

The Hann window is

$$w(t) = \frac{1}{2}\left[1 - \cos\left(2\pi\frac{t - t_0}{T - t_0}\right)\right], \qquad t_0 \le t \le T, \tag{10.36}$$

where

$$t_0 = \texttt{-rcs-start-time}, \qquad T = \texttt{-final-time}.$$

Samples before t_0 are ignored. The purpose of t_0 is to exclude:

1. startup transients produced by abruptly imposing a time-harmonic incident field,
2. early nonperiodic adjustment of the scattered field,
3. wave packets reflected by the PML before they have decayed.

For a quantitative RCS comparison, the RCS curve should be stable when `-rcs-start-time` is shifted moderately later and when `-rcs-every` is reduced.

10.8 DG Surface Quadrature in the Code

The RCS calculation uses the same distributed DG mesh and polynomial basis as the time integration. The core data structure is `RCSFaceSample`, built by `build_pec_sphere_face_samples`. For each owned boundary face tagged as the metallic sphere:

1. The code keeps only faces with boundary tag `METALLIC_SPHERE_PEC_BOUNDARY_ID` $= 10$.
2. The face must belong to an element owned by the local MPI rank.
3. The face interpolation nodes are copied from the DG trace map.
4. The face quadrature weights are obtained by applying the reference face mass matrix to the constant vector 1, then scaling by physical face area:

$$w_q^{(F)} = \frac{|F|}{|F_{\mathrm{ref}}|}\,(M_F\mathbf{1})_q\,. \tag{10.37}$$

5. The physical coordinates of each face node are reconstructed by mapping the reference tetrahedron coordinates to the physical tetrahedron.

6. The inner-boundary DG normal is flipped to obtain the PEC scatterer normal used in $\boldsymbol{J}_s = \boldsymbol{n} \times \boldsymbol{H}_{\mathrm{tot}}$.

At one time sample t_m, the local rank contribution is

$$\boldsymbol{I}_{h,\mathrm{rank}}(\widehat{\boldsymbol{r}}, t_m) = \sum_{F \in \mathcal{F}^{\mathrm{PEC}}_{\mathrm{rank}}} \sum_{q \in F} w_q^{(F)} \left[\boldsymbol{n}_F \times (\boldsymbol{H}_{\mathrm{scat},h}(\boldsymbol{x}_{Fq}, t_m) + \boldsymbol{H}_{\mathrm{inc}}(\boldsymbol{x}_{Fq}, t_m))\right] \mathrm{e}^{-\mathrm{i}k\widehat{\boldsymbol{r}}\cdot\boldsymbol{x}_{Fq}}. \quad (10.38)$$

The code accumulates this for every observation direction requested by the CLI. After multiplication by the Hann window and temporal Fourier phase, it adds the result to a rank-local complex accumulator:

$$\boldsymbol{A}_{\mathrm{rank}}(\widehat{\boldsymbol{r}}) \leftarrow \boldsymbol{A}_{\mathrm{rank}}(\widehat{\boldsymbol{r}}) + w_m \mathrm{e}^{\mathrm{i}\omega t_m} \boldsymbol{I}_{h,\mathrm{rank}}(\widehat{\boldsymbol{r}}, t_m). \quad (10.39)$$

The global accumulator is the MPI sum

$$\boldsymbol{A}(\widehat{\boldsymbol{r}}) = \sum_{\mathrm{rank}} \boldsymbol{A}_{\mathrm{rank}}(\widehat{\boldsymbol{r}}). \quad (10.40)$$

Finally,

$$\widehat{\boldsymbol{I}}_h(\widehat{\boldsymbol{r}}) = \frac{2}{W} \boldsymbol{A}(\widehat{\boldsymbol{r}}). \quad (10.41)$$

10.9 Algorithm Implemented in DiscoGMPI

The RCS path in `distributed_metallic_sphere_scattering.jl` can be summarized as follows.

1. Parse the scattering configuration: mesh, sphere radius, wavelength, frequency, amplitude, DG order, ESPRK order, CFL, PML parameters, flux choice, RCS angles, RCS sampling interval, and output directory.

2. Build the incident wave parameters:

$$\boldsymbol{d} = \widehat{\boldsymbol{z}}, \qquad \boldsymbol{p} = \widehat{\boldsymbol{x}}, \qquad k = 2\pi/\lambda, \qquad \omega = 2\pi f.$$

3. Load and partition the tetrahedral box-minus-sphere mesh.

4. Build the DG discretization and the incident-aware PEC boundary data.

5. Initialize the numerical unknown as the scattered field.

6. If RCS is enabled, build:

   - observation directions $\widehat{\boldsymbol{r}}(\theta, \phi)$,
   - transverse bases $\widehat{\boldsymbol{e}}_\theta$, $\widehat{\boldsymbol{e}}_\phi$,
   - owned PEC face samples,
   - complex current accumulators.

7. March the scattered field in time with the selected Poisson-bracket Maxwell/ESPRK path and PML source terms.

8. At every RCS sample time:

   - set the boundary-data time consistently,
   - reconstruct $\boldsymbol{H}_{\mathrm{tot},h} = \boldsymbol{H}_{\mathrm{scat},h} + \boldsymbol{H}_{\mathrm{inc}}$ on the PEC face nodes,
   - compute $\boldsymbol{J}_{s,h} = \boldsymbol{n} \times \boldsymbol{H}_{\mathrm{tot},h}$,

- integrate $\boldsymbol{J}_{s,h} \mathrm{e}^{-\mathrm{i}k\widehat{\boldsymbol{r}}\cdot\boldsymbol{x}}$ over the owned PEC faces,
- multiply by $w_m \mathrm{e}^{\mathrm{i}\omega t_m}$,
- add to the complex accumulator.

9. At final time, reduce the complex accumulators across MPI ranks.
10. Reconstruct $E^{\infty}_{\theta,h}$, $E^{\infty}_{\phi,h}$, and σ_h for every observation angle.
11. Write `diagnostics/rcs.csv`.

10.10 Observation Grid and CLI Controls

The production driver controls the RCS observation directions with:

`-rcs-theta-count`
: Number of polar angle samples between `-rcs-theta-min-degrees` and `-rcs-theta-max-degrees`.

`-rcs-theta-min-degrees, -rcs-theta-max-degrees`
: Minimum and maximum polar scattering angle in degrees.

`-rcs-phi-degrees`
: Comma-separated list of azimuthal angles in degrees.

`-rcs-every`
: Time-step interval between RCS surface integrations.

`-rcs-start-time`
: First physical time included in the Fourier extraction.

For a smooth E-plane RCS plot comparable to the common PEC-sphere benchmark, one should use, for example,

```
--rcs-theta-count=181 \
--rcs-theta-min-degrees=0 \
--rcs-theta-max-degrees=180 \
--rcs-phi-degrees=0
```

This gives one-degree angular spacing. Increasing `-rcs-theta-count` makes the RCS plot smoother but increases the RCS surface-integration cost approximately linearly. It does not change the DG time integration itself.

10.11 Output Files

The main RCS diagnostic file is

`<run-dir>/diagnostics/rcs.csv`.

It contains one row per observation direction. The key columns are:

Column	Meaning
`theta_degrees`, `phi_degrees`	Observation angles.
`dir_x`, `dir_y`, `dir_z`	Cartesian components of $\widehat{\boldsymbol{r}}$.
`abs_E_infinity`	$\lvert\boldsymbol{E}_h^{\infty}\rvert$.
`abs_Etheta`, `abs_Ephi`	$\lvert E_{\theta,h}^{\infty}\rvert$, $\lvert E_{\phi,h}^{\infty}\rvert$.
`rcs`	σ_h.
`rcs_db`	$10\log_{10}(\sigma_h)$, not normalized by πa^2.
`real_Etheta`, `imag_Etheta`	Complex $E_{\theta,h}^{\infty}$.
`real_Ephi`, `imag_Ephi`	Complex $E_{\phi,h}^{\infty}$.
`samples`	Number of nonzero-window RCS time samples.
`window_sum`	$W = \sum_m w_m$.
`rcs_start_time`, `rcs_end_time`	Fourier window endpoints.

The Python postprocessor writes:

- `diagnostics/mie_rcs_comparison.csv`,
- `tables/mie_rcs_comparison.tex`,
- `plots/mie_rcs_comparison.pdf`,
- `plots/mie_rcs_polar.pdf`,
- `plots/mie_rcs_theta0_phi_cut.pdf`.

The normalized plotted quantity is

$$10\log_{10}\left(\frac{\sigma}{\pi a^2}\right),$$

even though the raw `rcs_db` column in `rcs.csv` is the dB value of σ itself.

For the NASA PEC Almond workflow, `examples/postprocess_pec_almond_rcs.py` writes analogous comparison artifacts using the Austin normalization σ/λ_0^2, not $\sigma/(\pi a^2)$. The relevant outputs are:

- `diagnostics/pec_almond_rcs_comparison.csv`,
- LaTeX comparison tables under `tables/`,
- Cartesian comparison plots under `plots/`,
- optional PNG versions when `-write-png` is provided.

The almond comparison can be made against Austin simulation data `ref`, direct measurement data `meas-m1`, or scaled measurement data `meas-m2`.

10.12 Example RCS Figures

Figures 10.2 and 10.3 show the postprocessed output from

`output/validation_sequence/coarse_sphere_rcs_epw2_p4_transient_theta181/plots`.

The file `mie_rcs_theta0_phi_cut.pdf` is intentionally not included here because, at $\theta = 0°$, varying ϕ rotates the transverse basis but does not change the Cartesian observation direction. The two plots included below are the physically relevant forward-to-backward E-plane RCS views for the default $\phi = 0°$ cut.

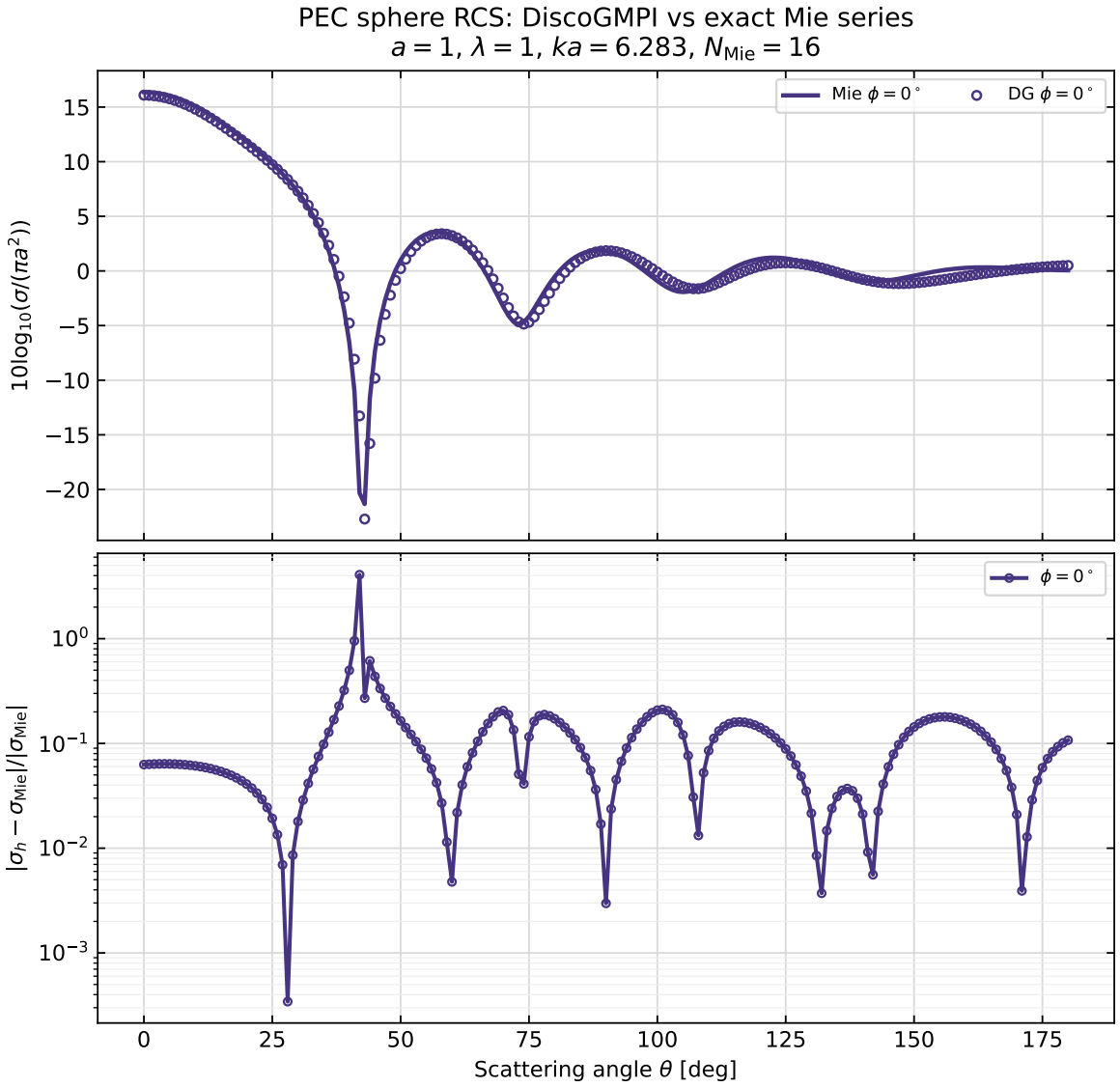


Figure 10.2: Cartesian RCS comparison for the `coarse_sphere_rcs_epw2_p4_transient_theta181` run. The upper panel compares $10\log_{10}(\sigma/(\pi a^2))$ for the DG near-to-far extraction and the exact PEC-sphere Mie series. The lower panel reports the relative RCS error.

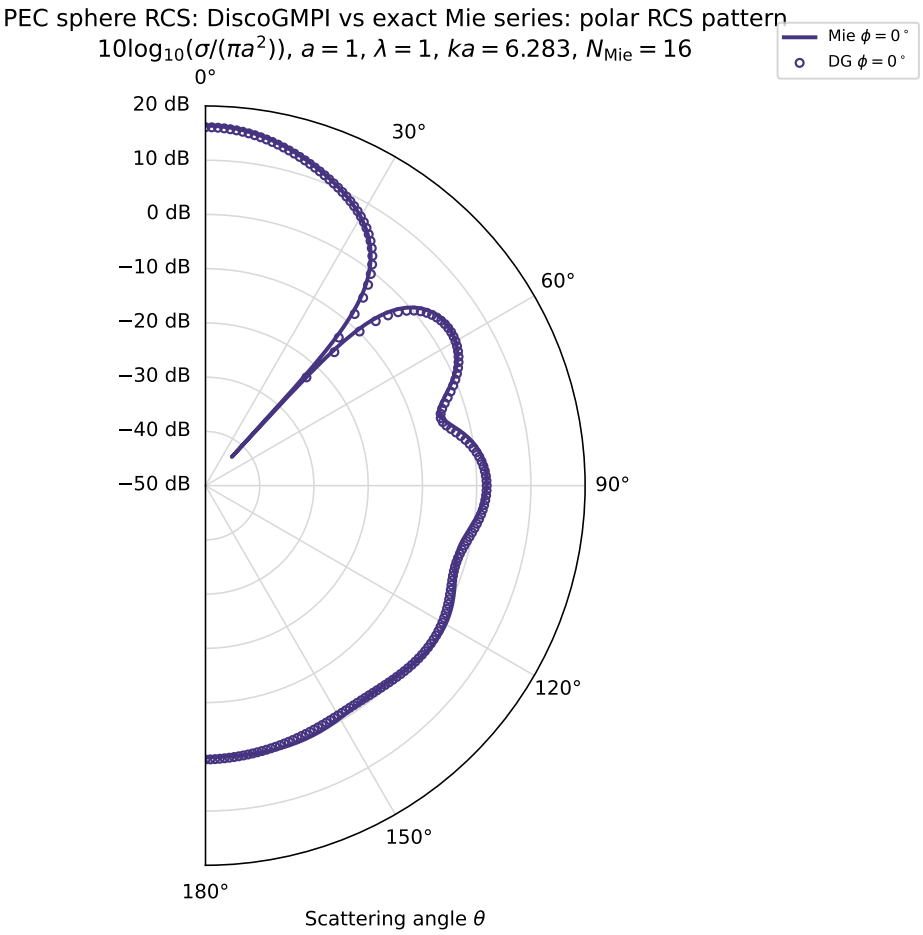


Figure 10.3: Polar RCS pattern for the same run. The angular coordinate is the DiscoGMPI polar scattering angle θ, with $\theta = 0^\circ$ denoting the forward $+z$ direction and $\theta = 180^\circ$ denoting backscatter. The radial tick labels are normalized dB values, $10\log_{10}(\sigma/(\pi a^2))$.

10.13 MN5 EPW4/P4 RCS Validation Run

The directory

`output/MN5/coarse_sphere_rcs_epw4_p4_transient_theta181`

contains an eight-rank MN5 execution of the metallic-sphere workflow. This is the higher-resolution run currently used as the main DiscoGMPI RCS validation case. The computation solves for the scattered field around a PEC sphere, adds the analytical incident wave for output and boundary data, extracts the time-harmonic scattered far field after the transient, and compares the result against the exact PEC-sphere Mie series.

Quantity	Value
Driver	`distributed_metallic_sphere_scattering` through the coarse RCS validation wrapper.
Mesh	`examples/meshes/metallic_sphere_scattering_epw4.vtk`.
Geometry	PEC sphere of radius $a = 1$ centered in the box $[-3, 3]^3$.
Absorbing layer	Cartesian nonlinear PML on the six outer box slabs, with width 1, $\sigma_{\max} = 12$, polynomial degree 2, and $a_{\mathrm{PML}} = 0.5$.
Incident wave	$+z$-propagating, x-polarized plane wave with amplitude 1, wavelength $\lambda = 1$, frequency $f = 1$, and $\epsilon = \mu = 1$.
Discretization	DG order $p = 4$, 35 nodes per tetrahedron, centered flux, ESPRK order 5, CFL 0.15.
Mesh size and time step	15673 global tetrahedra, $h_{\min} = 2.0491638407152006 \times 10^{-2}$, $\Delta t = 7.683934174297241 \times 10^{-4}$.
Time interval	$T = 12$, 15617 time steps.
MPI layout	8 ranks, owned-element imbalance 1.00051, with 1959–1960 owned elements per rank.
RCS extraction	Fourier window starts at $t = 5$, samples every 5 time steps, $\theta = 0^\circ, \ldots, 180^\circ$ with 181 angles, and $\phi = 0^\circ$.
Runtime	16477.35 s, approximately 4.58 hours.

The Mie comparison summary reports a successful validation gate for the available criteria: all 181 angular samples are finite and positive, the largest numerical RCS is 1.33865×10^2, and the Fourier window contains 1822 samples with a window sum of 910.992. For the PEC sphere with $ka = 2\pi$, the postprocessor used 16 Mie terms. The RMS relative error over the $\phi = 0^\circ$ forward-to-backward cut is 6.95×10^{-2}, the maximum relative error is 5.97×10^{-1}, the maximum dB error is 3.16 dB, and the maximum absolute error normalized by πa^2 is 7.25×10^{-1}. The largest relative errors occur near deep Mie nulls, where the exact RCS is small and relative normalization amplifies small absolute differences.

The same run also wrote time-series diagnostics and field visualization data. The final scattered energy was 7.0974, and the final PEC boundary residual diagnostic gave a relative RMS value 1.90×10^{-1}. This residual is a useful boundary-consistency diagnostic, but the RCS validation is primarily assessed by the far-field comparison against the Mie reference after the transient window begins.

10.14 NASA/Austin PEC Almond Benchmark

The NASA PEC Almond case in DiscoGMPI targets Problem Set IIIA of the Austin RCS Benchmark Suite. The object is a nontrivial perfectly electrically conducting almond with a sharp tip and a slender body. This geometry is much more demanding than the PEC sphere because there is no analytic Mie reference, the surface curvature varies strongly, and the tip can create very small tetrahedra if the imported surface mesh is not controlled.

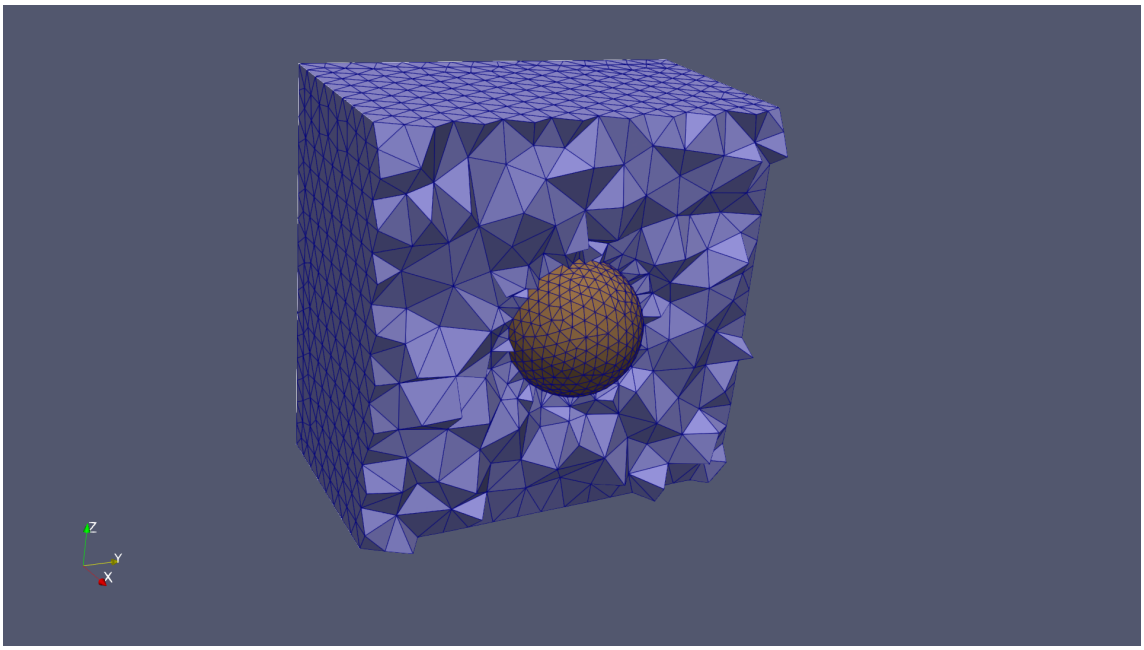


Figure 10.4: Mesh visualization for the MN5 EPW4/P4 PEC-sphere RCS run. The picture shows the tetrahedral box-minus-sphere discretization used by `examples/meshes/metallic_sphere_scattering_epw4.vtk`. The inner spherical boundary is the PEC scatterer on which the incident-aware boundary trace is imposed and from which the surface-current RCS integral is sampled. The surrounding Cartesian box provides the outer PML slabs. This figure is mainly a geometry and mesh-quality check: the Mie comparison is only meaningful if the spherical PEC surface and the near-target volume mesh are resolved consistently with the requested EPW4 discretization.

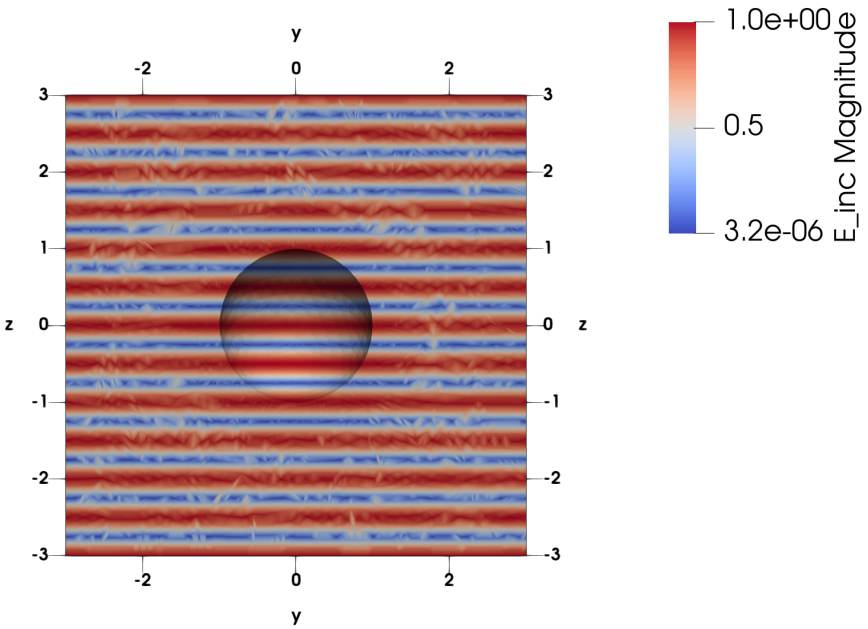


Figure 10.5: Incident-field visualization from the MN5 EPW4/P4 run. The plot shows the imposed $+z$-propagating, x-polarized plane wave used to construct the total field and the incident-aware PEC boundary condition.

10.14.1 Benchmark Definition

The benchmark object is a PEC almond proportional to the NASA benchmark almond dimensions. The smallest benchmark size used by the available reference data is

$$L = 9.936\,\text{in} = 0.2523744\,\text{m}. \tag{10.42}$$

The Austin data available in this repository cover the four frequency keys

Frequency key	Frequency	Free-space wavelength
`fx1`	3.5 GHz	8.5654988×10^{-2} m
`fx2`	5.125 GHz	5.8496089×10^{-2} m
`fx3`	7.0 GHz	4.2827494×10^{-2} m
`fx4`	10.25 GHz	2.9248045×10^{-2} m

The default DiscoGMPI almond workflow uses `fx1`, so

$$\lambda_0 = \frac{c_0}{3.5 \times 10^9} = 8.5654988 \times 10^{-2}\,\text{m}.$$

The Austin benchmark quantity of interest is monostatic RCS on the $\theta = 90^\circ$ aspect cut, with

$$0^\circ \le \phi \le 180^\circ.$$

The reference simulation files are sampled every 0.5°, giving 361 angular samples. The provided measurement files are sampled more densely, every 0.25°. The data-file columns are

$$\text{frequency (Hz)}, \quad \theta\ \text{(deg)}, \quad \phi\ \text{(deg)}, \quad \sigma_{\text{dBsm}}. \tag{10.43}$$

The reference files live in

`benchmark/AustinCEMBenchmarks/Problem_III-Almonds/IIIA-ReferenceData`.

Their naming convention is:

- `ref_rcs.III.A.s1.fxK.H.txt` and `ref_rcs.III.A.s1.fxK.V.txt`: reference simulation data;
- `meas.m1_rcs.III.A.s1.fxK.H.txt` and `meas.m1_rcs.III.A.s1.fxK.V.txt`: measurements of the $L = 9.936$ in almond;
- `meas.m2_rcs.III.A.s1.fxK.H.txt` and `meas.m2_rcs.III.A.s1.fxK.V.txt`: measurements of a two-times larger almond shifted by $10\log_{10} 4$ dB and by the corresponding frequency scaling.

Here `K` $\in \{1, 2, 3, 4\}$. The suffix `H` denotes HH polarization, and the suffix `V` denotes VV polarization.

10.14.2 Polarization and Normalization

The Austin notation uses $\sigma_{\phi\phi}$ for HH polarization and $\sigma_{\theta\theta}$ for VV polarization. The DiscoGMPI almond postprocessor maps these to the far-field components

$$\text{H/HH} \longleftrightarrow E_\phi^\infty, \qquad \text{V/VV} \longleftrightarrow E_\theta^\infty. \tag{10.44}$$

The raw DiscoGMPI RCS file stores total RCS and the component far-field magnitudes. The almond postprocessor reconstructs component RCS values using

$$\sigma_{\theta,h} = 4\pi \frac{|E_{\theta,h}^\infty|^2}{A^2}, \tag{10.45}$$

$$\sigma_{\phi,h} = 4\pi \frac{|E_{\phi,h}^\infty|^2}{A^2}. \tag{10.46}$$

The comparison plots use the dimensionless benchmark-style normalization

$$10 \log_{10} \left(\frac{\sigma}{\lambda_0^2} \right), \tag{10.47}$$

while the Austin text files store σ directly in dBsm. Therefore the postprocessor converts a reference dBsm value by

$$\sigma_{\lambda,\mathrm{dB}} = \sigma_{\mathrm{dBsm}} - 20 \log_{10}(\lambda_0). \tag{10.48}$$

10.14.3 DiscoGMPI Almond Workflow

The production frontend is

`examples/distributed_pec_almond_scattering.jl`.

It is intentionally thin: it reuses the generic PEC-scatterer Maxwell/PML/RCS driver in `examples/distributed_metallic_sphere_scattering.jl`. The almond frontend supplies:

- the default almond mesh, `examples/meshes/pec_almond_scattering_fx1_epw4.vtk`;
- the PEC boundary tag 10;
- the scatterer name `PEC almond`;
- radius 0, which disables the sphere-specific radius check;
- a physical frequency-to-wavelength conversion $\lambda_0 = c_0/f$;
- one-wavelength Cartesian PML width by default;
- RCS extraction enabled by default.

The solver itself remains nondimensional. With $\epsilon = \mu = 1$, the internal wave speed is 1. The physical benchmark frequency is used to set the numerical wavelength; for example,

$$f = 3.5\,\mathrm{GHz} \quad \Rightarrow \quad \lambda_0 = 8.5654988 \times 10^{-2}.$$

The normalized solver angular frequency is then $\omega = 2\pi/\lambda_0$.

The current Austin-style SLURM workflow is

`examples/run_nasa_almond_rcs.slurm`.

Its defaults run

- `fx1`: 3.5 GHz;
- DG order $p = 2$, ESPRK order 3;
- centered Poisson-bracket flux;
- final time $12\lambda_0$;
- RCS Fourier window start time $5\lambda_0$;
- $\theta = 90°$, $\phi = 0°, 0.5°, \ldots, 180°$;
- RCS sampling every 5 time steps;
- diagnostics every 250 time steps;
- ParaView output disabled unless `PARAVIEW_EVERY` is overridden.

The postprocessor is

`examples/postprocess_pec_almond_rcs.py`.

It reads `diagnostics/rcs.csv`, selects the requested angular cut, compares against Austin `ref`, `meas-m1`, or `meas-m2` files, and writes CSV, LaTeX, PNG, and PDF outputs in the run directory.

10.14.4 Current Default Mesh

The mesh generator is

`examples/meshes/generate_pec_almond_scattering_mesh.sh`

with geometry body

`examples/meshes/pec_almond_scattering.geo`.

The geometry is a Cartesian box with an inner PEC almond cavity. The six outer faces are tagged as PML boundaries and the almond surface is tagged as PEC:

Physical tag	Meaning
$1, \ldots, 6$	outer Cartesian box faces, used by the Cartesian PML;
10	inner PEC almond boundary;
7	air plus PML volume.

The current default EPW4 mesh `examples/meshes/pec_almond_scattering_fx1_epw4.vtk` has:

Quantity	Value
Points	9937
VTK cells	50331
Boundary triangles	16648
PEC almond triangles, tag 10	14300
Outer PML-boundary triangles, tags $1, \ldots, 6$	2348
Auxiliary vertex/line cells, tag -1	166
Tetrahedra, tag 7	33517
Domain lower corner	$(-0.297497176, -0.220090139, -0.187570926)$
Domain upper corner	$(0.297497176, 0.220090139, 0.187570926)$
Minimum tet edge	2.479743×10^{-5}
First-percentile tet edge	3.553260×10^{-4}
Median tet edge	1.170343×10^{-2}
$\min(3V/A)$	1.505260×10^{-6}
First-percentile $3V/A$	3.088514×10^{-5}
Median $3V/A$	6.517538×10^{-4}

Here $3V/A$ is the element-size measure used by `estimate_maxwell_dt`; it is the actual explicit-timestep limiter in the current driver.

Benchmark Mesh Figures

The Austin benchmark folder contains four PNG views of the NASA almond mesh. They are included in Figures 10.10, 10.11, 10.12, and 10.13 because they document the geometric features that control both accuracy and runtime:

- the global volume-mesh view shows the tetrahedral volume around the PEC cavity and the contrast between near-target and far-field mesh sizes;
- the xy-plane view shows the planform almond shape and the sharp downstream tip, which is the main source of small elements;
- the xz-plane view shows the slender longitudinal profile and the thin vertical thickness;
- the yz-plane view shows the cross-section, useful for checking whether the mesh resolves the transverse thickness without producing pathological slivers.

10.14.5 Reference Comparison and Error Metrics

Unlike the PEC sphere, the almond has no closed-form Mie-series reference. The quantitative reference is the benchmark data set. The DiscoGMPI postprocessor computes a row-wise dB error

$$e_{\mathrm{dB}}(\phi_j) = \sigma_{h,\lambda,\mathrm{dB}}(\phi_j) - \sigma_{\mathrm{ref},\lambda,\mathrm{dB}}(\phi_j). \tag{10.49}$$

The Austin benchmark also defines a thresholded dB error that discounts differences in regions where the reference RCS is more than 80 dB below the maximum reference level. For polarization $u \in \{\theta, \phi\}$, let

$$T_{u,\mathrm{dB}} = \max_{\phi} \sigma^{\mathrm{ref}}_{uu,\mathrm{dB}}(\phi) - 80. \tag{10.50}$$

Then the thresholded curves are

$$\sigma^{\mathrm{TH}}_{uu,\mathrm{dB}}(\phi) = \max\left(\sigma_{uu,\mathrm{dB}}(\phi), T_{u,\mathrm{dB}}\right), \tag{10.51}$$

and a benchmark-style average error is

$$\overline{e}^{\mathrm{TH}}_{uu,\mathrm{dB}} = \frac{1}{N_\phi} \sum_{j=1}^{N_\phi} \left| \sigma^{\mathrm{TH}}_{h,uu,\mathrm{dB}}(\phi_j) - \sigma^{\mathrm{TH}}_{\mathrm{ref},uu,\mathrm{dB}}(\phi_j) \right|. \tag{10.52}$$

This thresholding is important because deep RCS nulls can dominate relative or dB errors even when the absolute scattered field is small.

10.14.6 Interpretation and Current Limitations

The almond workflow should be interpreted differently from the sphere workflow:

- The sphere comparison is an exact-solution validation; the almond comparison is a benchmark-data validation.
- The sphere is geometrically smooth; the almond has sharp features, so mesh quality and the minimum $3V/A$ element size strongly affect runtime.
- The Austin data are monostatic aspect data. The current DiscoGMPI script samples the implemented RCS diagnostic on the $\theta = 90°$, $0 \le \phi \le 180°$ cut and compares the corresponding E^{∞}_{ϕ} or E^{∞}_{θ} component to the selected Austin reference. A strict monostatic aspect sweep requires the incident direction and observation direction to be swept consistently, or equivalently the target to be rotated against a fixed incident wave.
- Quantitative almond validation should therefore be reported with the exact run convention, mesh, frequency key, polarization, reference type, RCS window, and angular sampling.

10.14.7 Recommended Almond Commands

Generate the default almond mesh with:

```
bash examples/meshes/generate_pec_almond_scattering_mesh.sh
```

Run the default Austin-style RCS workflow on a SLURM system with:

```
sbatch examples/run_nasa_almond_rcs.slurm
```

Useful overrides are:

```
sbatch --export=ALL,ORDER=4,ESPRK_ORDER=5,FINAL_TIME=1.027859856,\
RCS_START_TIME=0.42827494,PARAVIEW_EVERY=500 \
  examples/run_nasa_almond_rcs.slurm
```

For an interactive MPI run without the SLURM wrapper:

```
mpiexec -n 8 -genv OPENBLAS_NUM_THREADS 1 julia --project=. \
  examples/distributed_pec_almond_scattering.jl \
  --mesh=examples/meshes/pec_almond_scattering_fx1_epw4.vtk \
  --frequency-hz=3.5e9 \
  --order=2 --esprk-order=3 \
  --final-time=1.027859856 \
  --rcs-start-time=0.42827494 \
  --rcs-every=5 \
  --rcs-theta-count=1 \
  --rcs-theta-min-degrees=90 \
  --rcs-theta-max-degrees=90 \
  --rcs-phi-degrees="$(python3 -c 'print(",".join(f"{0.5*i:.1f}" for i in range
    (361)))')" \
  --diagnostics-every=250 \
  --paraview-every=0 \
  --output-dir=output/validation_sequence/pec_almond_rcs_fx1_p2_theta90_phi0_180
```

Then compare against both HH and VV Austin reference curves:

```
python3 examples/postprocess_pec_almond_rcs.py \
  --run-dir=output/validation_sequence/pec_almond_rcs_fx1_p2_theta90_phi0_180 \
  --frequency-hz=3.5e9 \
  --frequency-key=fx1 \
  --reference-kind=ref \
  --polarization=both \
  --dg-angle=phi \
  --theta-filter=90 \
  --write-png
```

10.15 Exact PEC-Sphere Mie Reference

For a PEC sphere, the exact frequency-domain solution is given by the Mie series. The postprocessor evaluates the scattering amplitude functions $S_1(\theta)$ and $S_2(\theta)$ using the PEC Mie coefficients. With size parameter

$$x = ka,$$

the scalar Riccati-Bessel functions are

$$\psi_n(x) = x j_n(x), \qquad \xi_n(x) = x h_n^{(1)}(x),$$

and the PEC coefficients used by the code are

$$a_n = -\frac{\psi_n'(x)}{\xi_n'(x)}, \qquad b_n = -\frac{\psi_n(x)}{\xi_n(x)}. \tag{10.53}$$

The angular functions are

$$\pi_1(\mu) = 1, \tag{10.54}$$

$$\pi_2(\mu) = 3\mu, \tag{10.55}$$

$$\pi_n(\mu) = \frac{2n-1}{n-1}\mu\,\pi_{n-1}(\mu) - \frac{n}{n-1}\pi_{n-2}(\mu), \qquad n \geq 3, \tag{10.56}$$

$$\tau_n(\mu) = n\mu\,\pi_n(\mu) - (n+1)\pi_{n-1}(\mu), \qquad \mu = \cos\theta. \tag{10.57}$$

The scattering amplitudes are

$$S_1(\theta) = \sum_{n=1}^{N} \frac{2n+1}{n(n+1)} \left[a_n \pi_n(\cos\theta) + b_n \tau_n(\cos\theta) \right], \tag{10.58}$$

$$S_2(\theta) = \sum_{n=1}^{N} \frac{2n+1}{n(n+1)} \left[a_n \tau_n(\cos\theta) + b_n \pi_n(\cos\theta) \right]. \tag{10.59}$$

For the DiscoGMPI default incident wave, the postprocessor maps these amplitudes to the same $(\widehat{\boldsymbol{e}}_\theta, \widehat{\boldsymbol{e}}_\phi)$ basis as the numerical RCS:

$$E^{\infty}_{\theta,\mathrm{Mie}} = \frac{S_2(\theta)\cos\phi}{k}, \qquad E^{\infty}_{\phi,\mathrm{Mie}} = -\frac{S_1(\theta)\sin\phi}{k}. \tag{10.60}$$

Then

$$\sigma_{\mathrm{Mie}}(\theta,\phi) = 4\pi \left(|E^{\infty}_{\theta,\mathrm{Mie}}|^2 + |E^{\infty}_{\phi,\mathrm{Mie}}|^2 \right). \tag{10.61}$$

The automatic truncation order is

$$N = \max\left(8,\ \left\lceil x + 4x^{1/3} + 2 \right\rceil\right). \tag{10.62}$$

10.16 Validation Strategy

A correct RCS workflow must pass several levels of validation.

10.16.1 Boundary Residual

Before trusting RCS data, verify the incident-aware PEC boundary condition:

$$R_{\mathrm{PEC}} = \left\| \boldsymbol{n} \times (\boldsymbol{E}_{\mathrm{scat},h} + \boldsymbol{E}_{\mathrm{inc}}) \right\|_{L^2(\Gamma_{\mathrm{PEC}})}. \tag{10.63}$$

The validation driver `examples/validate_pec_sphere_boundary_residual.jl` writes `diagnostics/pec_boundary_residual.csv`. A large residual means the surface current is being evaluated on a field that does not satisfy the PEC condition accurately.

For the NASA PEC Almond, use `examples/validate_pec_almond_boundary_residual.jl`. It evaluates the same residual on the almond surface tag 10. The almond residual is especially useful because errors near the sharp tip can contaminate the surface current used for RCS extraction.

10.16.2 Qualitative Near Field

Inspect $\boldsymbol{E}_{\mathrm{scat}}$, $\boldsymbol{H}_{\mathrm{scat}}$, $\boldsymbol{E}_{\mathrm{tot}}$, and $\boldsymbol{H}_{\mathrm{tot}}$ in ParaView. The production driver can write:

- $\boldsymbol{E}_{\mathrm{scat}}, \boldsymbol{H}_{\mathrm{scat}}$,
- $\boldsymbol{E}_{\mathrm{inc}}, \boldsymbol{H}_{\mathrm{inc}}$,
- $\boldsymbol{E}_{\mathrm{tot}}, \boldsymbol{H}_{\mathrm{tot}}$,
- $|\boldsymbol{E}_{\mathrm{scat}}|, |\boldsymbol{H}_{\mathrm{scat}}|$.

For the default plane wave, the components E_x and H_y are the most direct comparison with the metallic-sphere benchmark figures.

10.16.3 Mie Comparison

After a run finishes, use

```
python3 examples/postprocess_metallic_sphere_mie_rcs.py \
  --run-dir <run-dir>
```

The comparison figure should be interpreted as:

- top panel: numerical and Mie $10\log_{10}(\sigma/(\pi a^2))$;
- bottom panel:

$$\frac{|\sigma_h - \sigma_{\mathrm{Mie}}|}{|\sigma_{\mathrm{Mie}}|}.$$

For quantitative validation, repeat the run with:

1. later `-rcs-start-time`,
2. smaller `-rcs-every`,
3. more angular samples,
4. finer sphere geometry,
5. larger polynomial order,
6. lower PML reflection.

A reliable RCS estimate should not change significantly under these variations.

10.16.4 Austin Almond Reference Comparison

For the almond, use:

```
python3 examples/postprocess_pec_almond_rcs.py \
  --run-dir <run-dir> \
  --frequency-key fx1 \
  --reference-kind ref \
  --polarization both \
  --dg-angle phi \
  --theta-filter 90
```

This comparison is not an exact-solution validation. It is a benchmark-data comparison against Austin simulation or measurement files. Report at least:

- frequency key and physical frequency;
- reference kind, `ref`, `meas-m1`, or `meas-m2`;
- polarization, H/HH or V/VV;
- angular cut and sampling;
- RCS window start time and number of Fourier samples;
- mesh name, DG order, and MPI rank count;
- mean and maximum dB error, preferably with the Austin thresholded metric when reporting benchmark performance.

10.17 Recommended Commands

A high-order EPW2/EPW4 smoke or development run can be launched with:

```
mpiexec -n 2 -genv OPENBLAS_NUM_THREADS 1 julia --project=. \
  examples/validate_coarse_sphere_rcs.jl \
  --mesh=examples/meshes/metallic_sphere_scattering_epw2.vtk \
  --order=4 --esprk-order=5 \
  --final-time=12.0 --rcs-start-time=5.0 \
  --rcs-every=5 --diagnostics-every=250 \
  --rcs-theta-count=181 \
  --rcs-theta-min-degrees=0 \
  --rcs-theta-max-degrees=180 \
  --rcs-phi-degrees=0 \
  --min-max-rcs=0 \
  --output-dir=output/validation_sequence/sphere_rcs_epw2_p4_theta181
```

Then postprocess:

```
python3 examples/postprocess_metallic_sphere_mie_rcs.py \
  --run-dir output/validation_sequence/sphere_rcs_epw2_p4_theta181
```

For a publication-quality RCS curve, EPW2 is usually too coarse geometrically. Use at least EPW3 or EPW4, enough runtime for a steady monochromatic response, and an RCS window that excludes startup transients.

For the NASA PEC Almond benchmark, prefer the SLURM workflow in `examples/run_nasa_almond_rcs.slurm`; it sets the Austin $\theta = 90°$, $0° \leq \phi \leq 180°$ angular cut and runs the almond postprocessor automatically. See Section 10.14 for the full command sequence and caveats.

10.18 Common Failure Modes

Too few points in the RCS curve.
Increase `-rcs-theta-count`. The postprocessor only plots the angles present in `diagnostics/rcs.csv`.

Mie and DG curves are mirrored left-to-right.
Check the near-to-far phase sign. For the convention $\cos(kz - \omega t)$, the surface integral uses $\exp(-\mathrm{i}k\hat{\boldsymbol{r}} \cdot \boldsymbol{x})$.

Large error near deep Mie nulls.
Relative error is sensitive when σ_{Mie} is very small. Inspect absolute normalized error and dB error as well.

Good boundary residual but poor RCS.
Possible causes are insufficient sphere geometry resolution, PML reflection, poor Fourier window, too few periods in the accumulation window, or too coarse angular sampling.

Good early RCS but later contamination.
Reflections from the PML or outer boundary may be returning to the sphere. Increase PML quality, enlarge the buffer, or end the Fourier window before reflections return.

No RCS rows.
RCS was disabled, the run failed before final output, no faces were tagged with the metallic sphere boundary ID, or the Fourier window contained no nonzero samples.

Confusion between θ and ϕ.
In DiscoGMPI, θ is the polar scattering angle from $+z$, and ϕ is the azimuthal angle. The standard E-plane sphere plot is $\phi = 0$ with $\theta \in [0°, 180°]$.

Almond HH/VV curves appear swapped.
Check the polarization mapping. In the almond postprocessor, `H` means HH and uses E_ϕ^∞, while `V` means VV and uses E_θ^∞.

Almond comparison has no matching reference rows.
Check `-frequency-key`, `-reference-kind`, and the selected angular cut. The Austin reference simulation files use $\theta = 90°$ and $\phi = 0°, 0.5°, \ldots, 180°$.

Almond run is much slower than expected.
Inspect the mesh-size diagnostics, especially $\min(3V/A)$. A few sliver tetrahedra near the almond tip can reduce the explicit time step by orders of magnitude even when the average edge length looks reasonable.

Good almond curve shape but persistent dB offset.
Verify the normalization. The almond plots use σ/λ_0^2; the sphere plots use $\sigma/(\pi a^2)$. Mixing these normalizations creates a constant vertical shift in dB.

10.19 Checklist for a Quantitative RCS Study

1. Verify the mesh: PEC boundary tag 10, adequate target surface resolution, adequate PML/buffer region, and no pathological $\min(3V/A)$ outliers.
2. Verify the incident wave: direction $+z$, polarization x, wavelength, frequency, amplitude, ϵ, and μ.
3. Verify PEC residual: $\|\boldsymbol{n} \times \boldsymbol{E}_{\text{tot}}\|_{L^2(\Gamma_{\text{PEC}})}$.
4. Run long enough for a periodic scattered field.
5. Set `-rcs-start-time` after the startup transient.
6. Use enough angular samples, e.g. `-rcs-theta-count=181` or `361`.
7. Compare against the corrected Mie postprocessor.
8. Repeat with a later `-rcs-start-time`.
9. Repeat with a smaller `-rcs-every`.
10. Repeat on a finer mesh or with a higher-order discretization.
11. Confirm that the RCS curve and error metrics stabilize.
12. For the almond, verify the benchmark convention separately: $\theta = 90°$, $\phi \in [0°, 180°]$, H/HH mapped to E_ϕ^∞, V/VV mapped to E_θ^∞, and σ/λ_0^2 normalization.

10.20 Code Map

The main implementation locations are:

File or function	Role
`examples/distributed_metallic_sphere_scattering.jl`	Production scattering and RCS driver.
`RCSObservationDirection`	Stores (θ, ϕ), $\widehat{\boldsymbol{r}}$, $\widehat{\boldsymbol{e}}_\theta$, $\widehat{\boldsymbol{e}}_\phi$.
`RCSFaceSample`	Stores owned PEC face nodes, weights, coordinates, and normals.
`build_pec_sphere_face_samples`	Extracts tagged sphere faces and flips the inner-boundary normal.
`accumulate_rcs_sample!`	Computes the windowed surface-current Fourier contribution.
`global_rcs_accumulator`	MPI reduction of complex accumulators.
`far_field_from_surface_current`	Applies (10.32).
`write_rcs_results`	Writes `diagnostics/rcs.csv`.
`src/MaxwellExactSolutions.jl`	Incident plane-wave definitions.
`examples/validate_coarse_sphere_rcs.jl`	Coarse validation gate and Julia Mie comparison.
`examples/postprocess_metallic_sphere_mie_rcs.py`	Publication-style Mie comparison plots and tables.
`examples/distributed_pec_almond_scattering.jl`	NASA/Austin PEC Almond frontend over the generic PEC-scatterer driver.
`examples/meshes/pec_almond_scattering.geo`	Box-minus-almond Gmsh geometry with PML outer tags and PEC almond tag.
`examples/meshes/generate_pec_almond_scattering_mesh.sh`	Reproducible almond mesh-generation wrapper.
`examples/validate_pec_almond_boundary_residual.jl`	Almond PEC residual gate.
`examples/validate_pec_almond_rcs_smoke.jl`	Short almond RCS smoke gate for finite, positive, angle-complete output.
`examples/postprocess_pec_almond_rcs.py`	Austin benchmark comparison for HH/VV almond RCS data.
`examples/run_nasa_almond_rcs.slurm`	End-to-end SLURM workflow: run almond RCS and postprocess against Austin data.
`benchmark/AustinCEMBenchmarks/Problem_III-Almonds`	Local Austin benchmark description, reference data, and mesh figures.

10.21 Summary

DiscoGMPI computes PEC-target RCS by solving the scattered-field Maxwell problem in the time domain, reconstructing the total magnetic field on the metallic surface, integrating the equivalent PEC surface current with the outgoing far-field phase $\exp(-\mathrm{i}k\widehat{\boldsymbol{r}} \cdot \boldsymbol{x})$, extracting the phasor at the incident frequency with a Hann-windowed Fourier average, and normalizing the far-field amplitude according to

$$\sigma = 4\pi |\boldsymbol{E}^\infty|^2 / A^2.$$

The procedure is sensitive to phase convention, angular convention, surface normal orientation, mesh geometry, PML reflections, and Fourier-window selection. The corrected Mie postprocessor provides the primary quantitative reference for the PEC sphere, but a trustworthy comparison requires a resolved mesh and a transient-free Fourier window. The NASA PEC Almond workflow uses the same near-to-far machinery, but validation is against Austin benchmark data rather than an exact solution; the important additional ingredients are the HH/VV component mapping, σ/λ_0^2 normalization, the $\theta = 90^\circ$ azimuthal cut, and careful control of the almond mesh near the sharp tip.

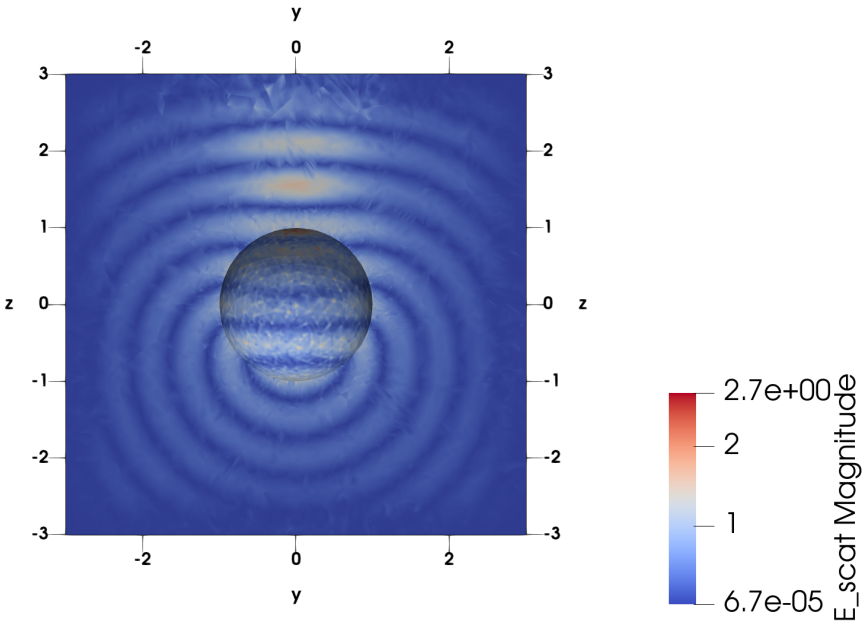


Figure 10.6: Scattered-field visualization from the MN5 EPW4/P4 run. The outgoing pattern around the sphere is the field used by the near-to-far RCS extraction after the time-windowed Fourier transform.

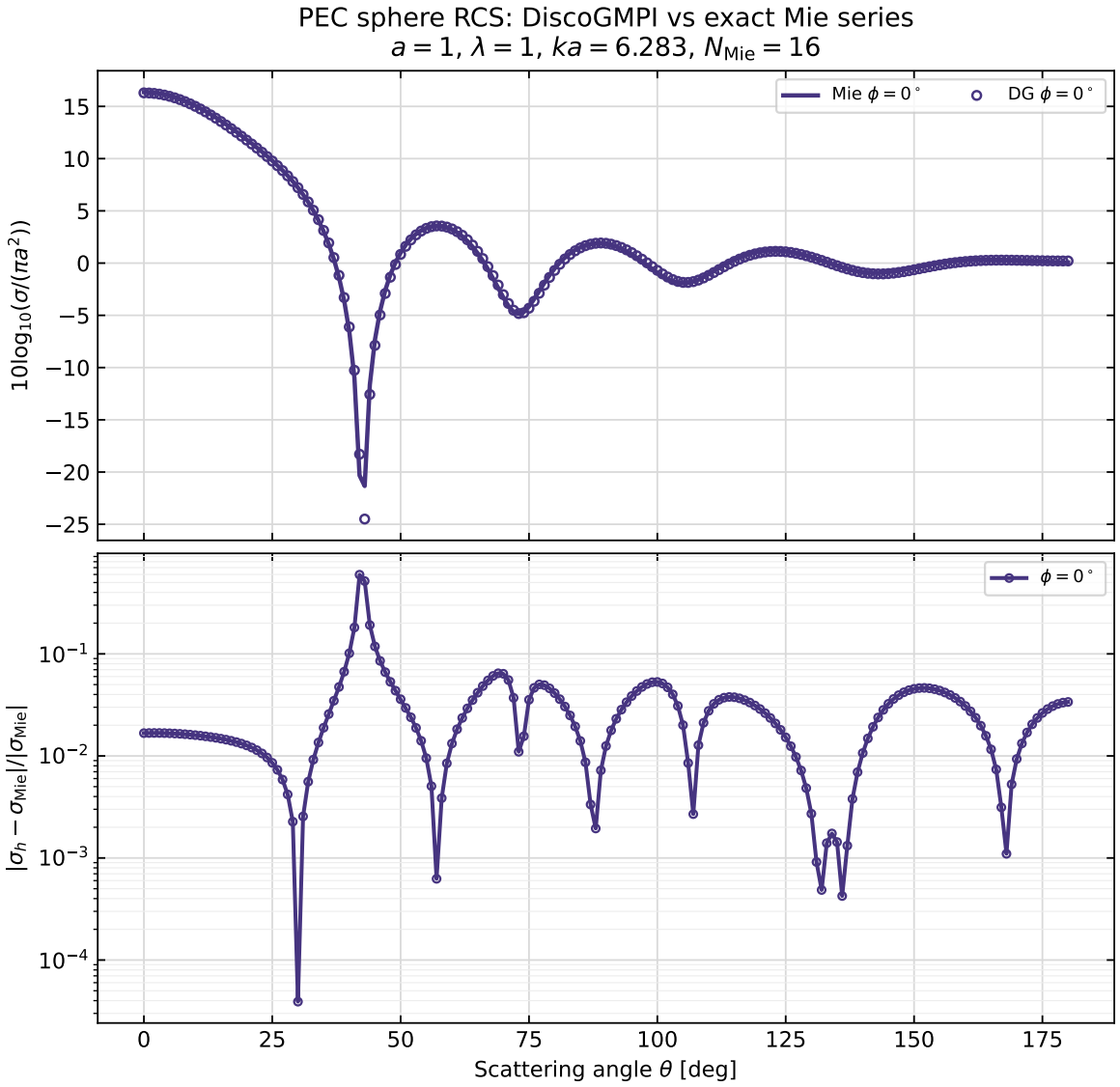


Figure 10.7: MN5 EPW4/P4 Cartesian RCS comparison. The upper panel compares the DG near-to-far result with the exact Mie curve for $10\log_{10}(\sigma/(\pi a^2))$. The lower panel reports the relative error; the visible peaks correspond mainly to angular neighborhoods of Mie nulls.

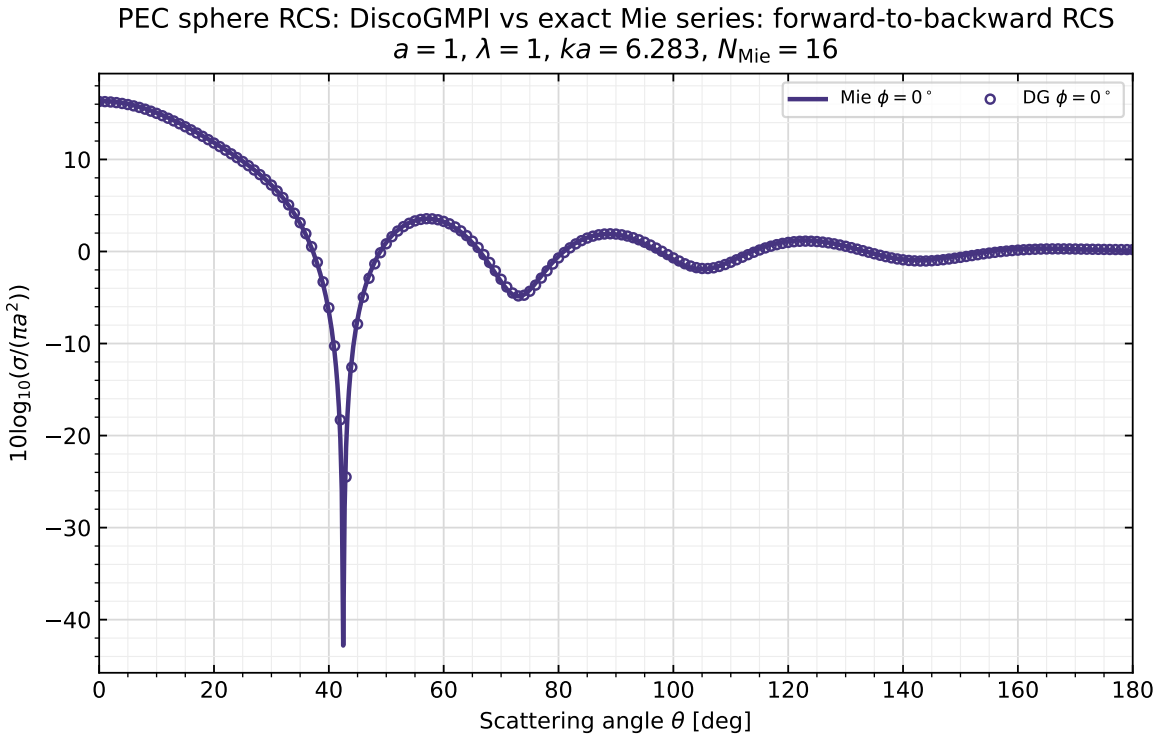


Figure 10.8: Forward-to-backward RCS cut for the MN5 EPW4/P4 run. Here $\theta = 0°$ is the forward $+z$ direction and $\theta = 180°$ is the backscatter direction.

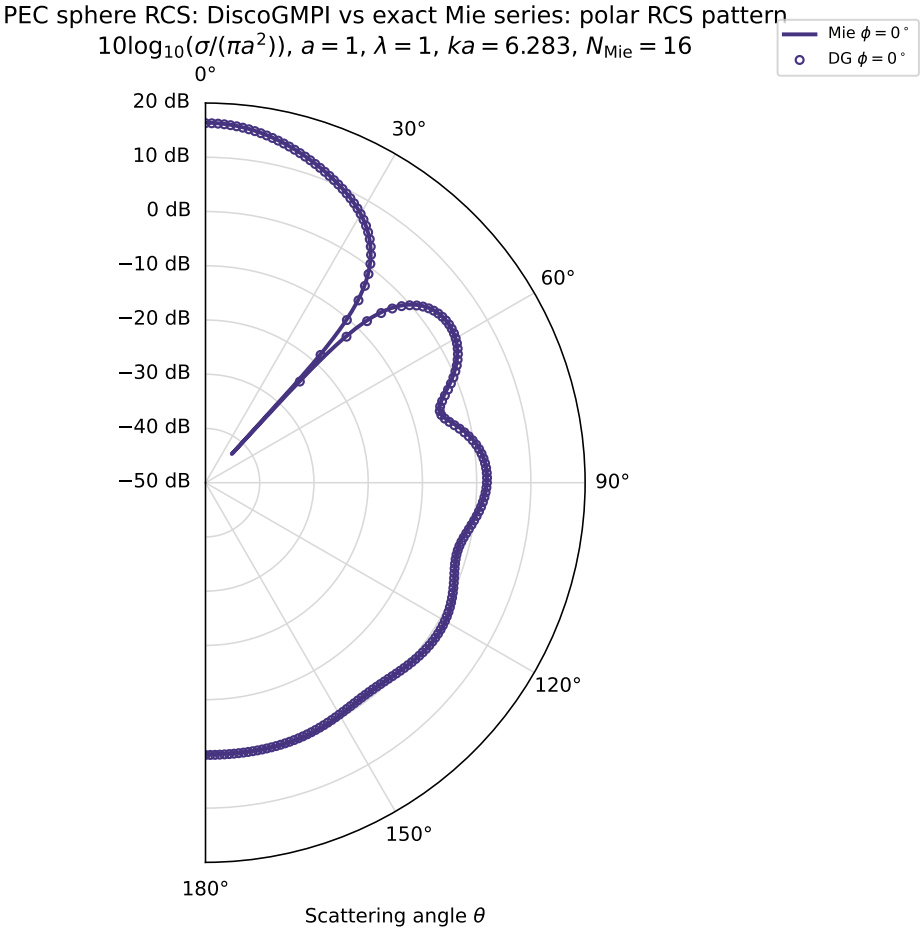


Figure 10.9: Polar representation of the MN5 EPW4/P4 RCS pattern. The radial coordinate is the normalized dB RCS, $10\log_{10}(\sigma/(\pi a^2))$.

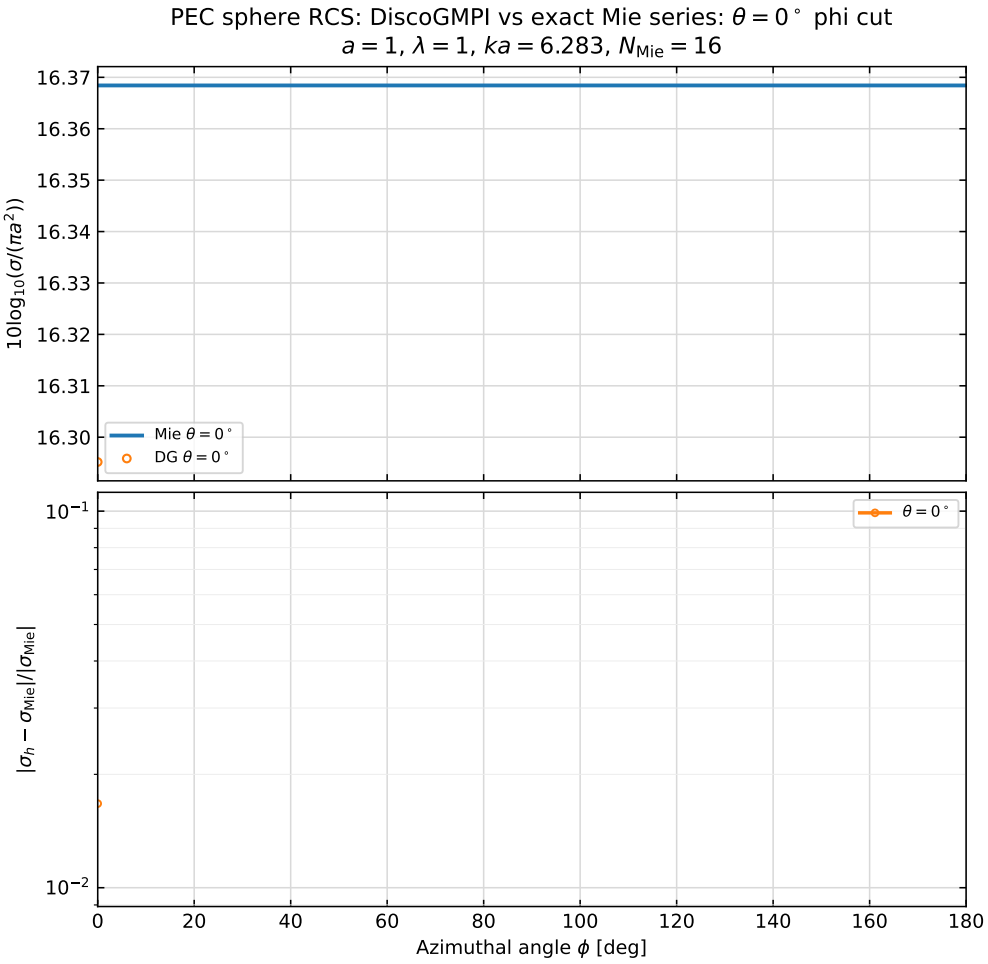


Figure 10.10: $\theta = 0°$ azimuthal diagnostic produced by the MN5 EPW4/P4 postprocessor. This run requested only $\phi = 0°$, so the DG data contain one azimuth sample. The figure is therefore a bookkeeping and orientation check, not a full ϕ-sweep validation.

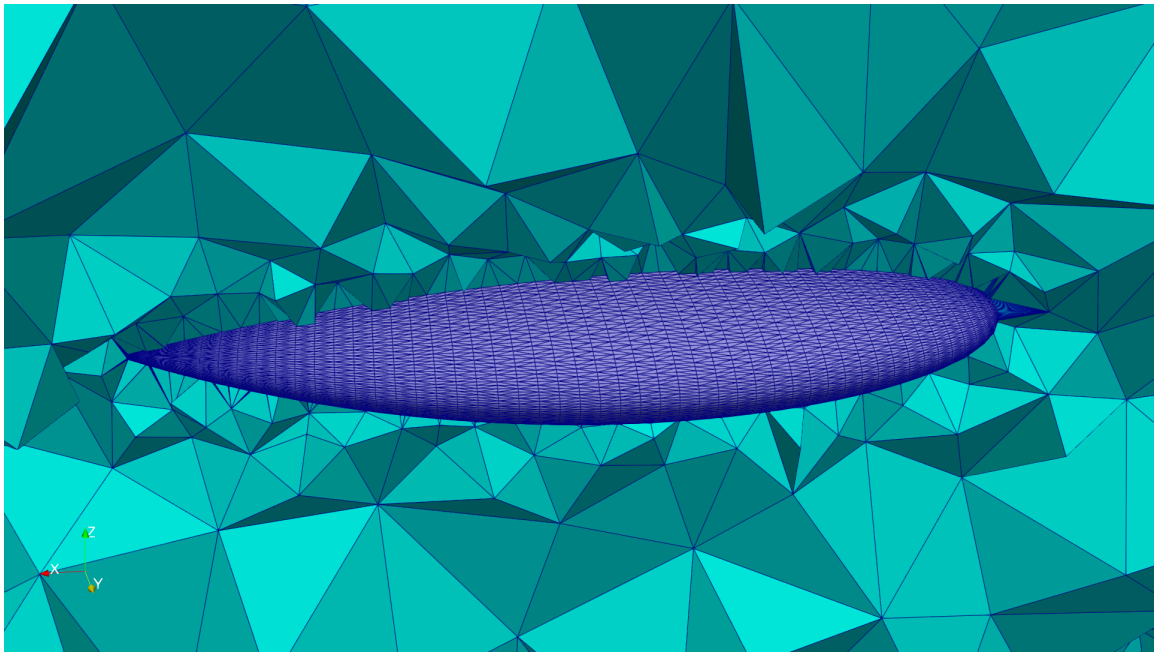


Figure 10.11: NASA PEC almond volume-mesh view from the Austin benchmark mesh folder. The almond is embedded as an inner PEC boundary in a tetrahedral volume mesh. This view is useful for inspecting near-target refinement, surrounding air volume, and the transition to coarser elements away from the scatterer.

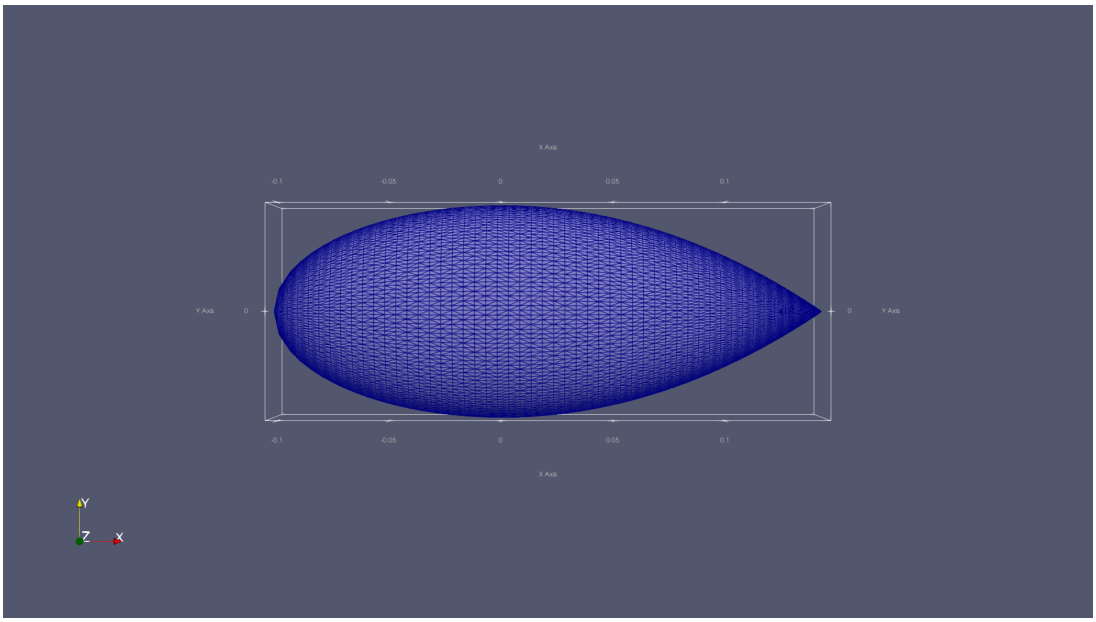


Figure 10.12: xy-plane PNG view of the Austin PEC Almond benchmark mesh. This planform view shows the rounded nose and sharp downstream tip, which are the main geometric features that drive local refinement and timestep cost.

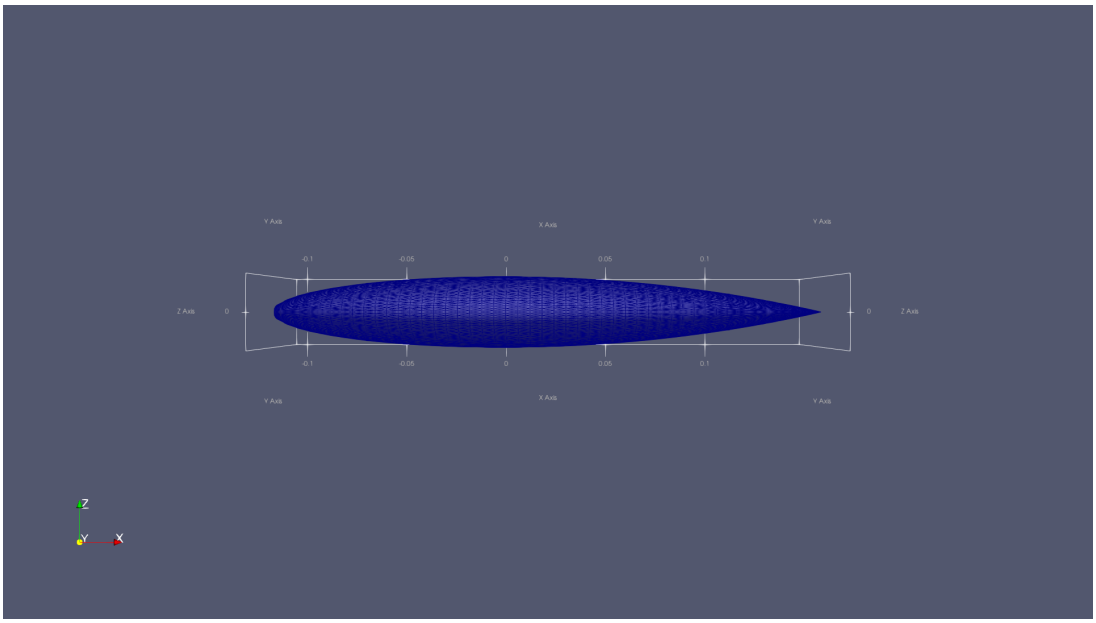


Figure 10.13: xz-plane PNG view of the Austin PEC Almond benchmark mesh. This longitudinal view shows the slender body profile and vertical thickness that must be resolved by the near-surface tetrahedra.

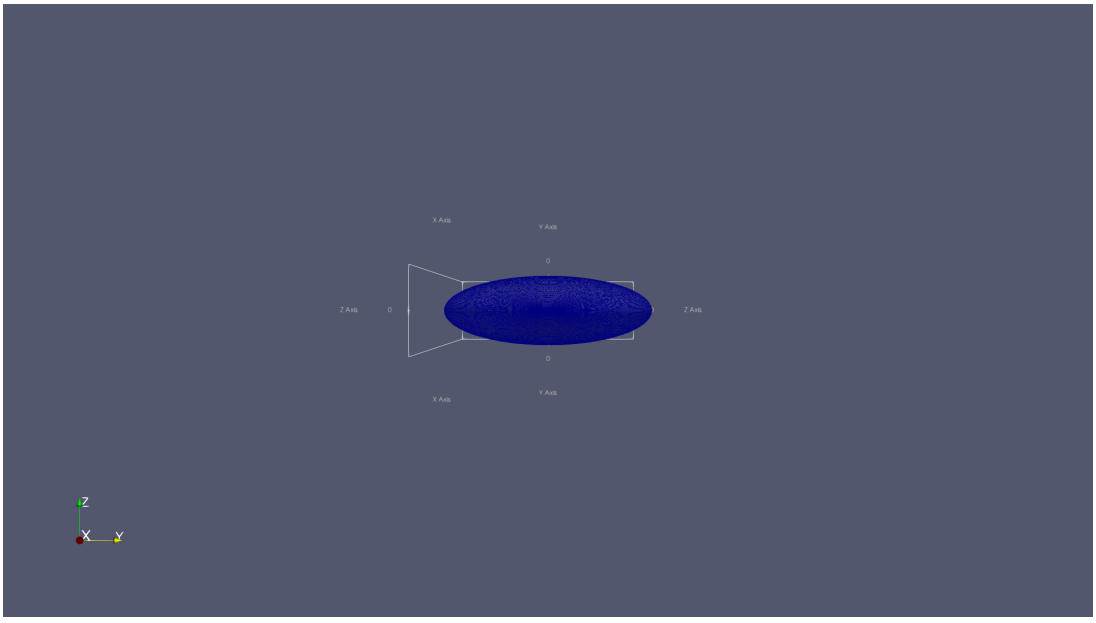


Figure 10.14: yz-plane PNG view of the Austin PEC Almond benchmark mesh. This transverse section is useful for checking the thin almond thickness and for identifying cross-sectional mesh-quality issues before a long RCS run.

Chapter 11

Testing

11.1 Package tests

Run the non-MPI test target with:

```
julia --project=. -e 'using Pkg; Pkg.test()'
```

The package suite covers loading, physical weak derivatives, Maxwell volume and surface operators, formulations, backend dispatch, time integrators, energy diagnostics, and distributed-mesh consistency constructed without an MPI launcher.

11.2 MPI face permutation test

```
mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
```

This test uses two tetrahedra whose shared face has different local vertex ordering. It verifies all face-node values after MPI exchange, including quadratic edge nodes.

11.3 MPI METIS partition tests

```
mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
```

These tests check owned and ghost counts, symmetric neighbor tables, deterministic communication ordering, interface orientation, and generic trace exchange.

11.4 Distributed Maxwell equivalence tests

```
mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
```

The two-rank test compares distributed and serial results for:

- Hesthaven–Warburton central flux;
- Hesthaven–Warburton upwind flux;
- Poisson-bracket formulation;

- PEC and PMC physical boundaries;
- exchanged ghost traces;
- zero ghost residuals;
- global energy;
- one explicit RK3 step;
- one partitioned symplectic RK2 step.
- rank-local mesh persistence and reload;
- checkpoint/restart field round trips;
- run and partition metadata generation.

The four-rank chain test exercises ranks with two neighbors and verifies upwind right-hand sides, traces, ghost residuals, and RK3 stepping. The file also verifies that root input errors propagate collectively without an MPI deadlock.

11.5 Nonlinear PML tests

The package test suite checks the nonlinear source against the component equations in Appendix A of the paper, verifies the nonpositive pointwise energy contribution, exercises zero-field regularization, and confirms that a zero damping profile reproduces the ordinary Maxwell right-hand side.

The distributed test is:

```
mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl
```

It verifies the Poisson-bracket PML source on owned elements, zero ghost residuals, and agreement between serial and distributed RK4 updates. The serial test also verifies nonlinear-PML energy decay.

11.6 Recommended pre-commit sequence

```
julia --project=. -e 'using Pkg; Pkg.test()'
mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl
mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl
```

Chapter 12

Benchmarking

12.1 Single benchmark

```
julia --project=. --threads=8 \
  benchmark/solver/benchmark_backends.jl \
  --formulation=hw-upwind \
  --cells=6 \
  --order=3 \
  --seconds=3
```

Select the Poisson-bracket operator with `-formulation=pb`. The benchmark reports minimum, median, and mean times, allocations, memory, speedup, efficiency, and serial-versus-threaded right-hand-side error.

12.2 Thread scaling sweep

```
THREAD_COUNTS="1 2 4 8" \
  bash benchmark/solver/run_thread_scaling.sh \
  --formulation=hw-upwind --cells=6 --order=3
```

Outputs:

- `benchmark/solver/thread_scaling_raw.txt`;
- `benchmark/solver/thread_scaling.csv`.

12.3 Plot scaling data

```
python3 benchmark/solver/plot_thread_scaling.py
```

The script writes:

- `benchmark/solver/thread_speedup.png`;
- `benchmark/solver/thread_efficiency.png`.

Install Matplotlib separately if it is not available:

```
python3 -m pip install matplotlib
```

12.4 MPI performance measurements

The MPI scaling harness runs strong and weak right-hand-side scaling sweeps:

```
MPI_RANKS="1 2 4 8" \
SCALING_MODES="strong weak" \
  bash benchmark/distributed/run_mpi_scaling.sh \
  --setup=cache --cells=6 --order=3 --iterations=50 --warmup=5
```

Strong scaling keeps `-cells` fixed. Weak scaling uses `round(cells * ranks^(1/3))` cells per axis so the element count grows approximately linearly with MPI ranks. The benchmark warms up the RHS, places a barrier immediately before timing, reduces maximum elapsed times across ranks, and reports separate columns for halo exchange, RHS assembly, ghost zeroing, owned elements, interface faces, ghost elements, and message sizes.

Outputs:

- `benchmark/distributed/mpi_scaling_raw.txt`;
- `benchmark/distributed/mpi_scaling.csv`;
- per-case `run_metadata.toml` and `partition_metadata.csv` under `benchmark/distributed/output`.

Chapter 13

Output and Visualization

13.1 Serial VTU field output

The stationary PEC examples contain helper functions that write either:

- cell-centered fields for order zero; or
- discontinuous vertex fields for order one and above.

The discontinuous writer duplicates tetrahedron vertices per element. This is intentional: a DG solution may have two different traces at the same geometric vertex.

13.2 Partition output

`write_partition_vtk` writes the complete mesh with a partition cell array. `write_partition_piece_vtks` writes one owned-element mesh per partition. These are mesh visualization utilities; they do not write the distributed Maxwell solution.

13.3 Distributed mesh files

The first production run may prepare one owned-plus-halo mesh shard per MPI rank:

```
prepare_distributed_mesh_partition(
    root_mesh,
    root_partition,
    "mesh_cache";
    comm = MPI.COMM_WORLD,
)
```

Later runs bypass global mesh loading and root distribution:

```
distributed_dg = build_distributed_dg_from_partition(
    "mesh_cache",
    order;
    comm = MPI.COMM_WORLD,
)
```

The directory contains `mesh_manifest.toml` and one checksummed binary piece per rank. All ranks must use the same shared filesystem and MPI rank count used when the cache was prepared.

13.4 Checkpoint and restart

```
write_distributed_checkpoint(
    "checkpoints/step00000100",
    U,
    distributed_dg;
    step = 100,
    time = 0.1,
    dt = 1.0e-3,
)

U, state = load_distributed_checkpoint(
    "checkpoints/step00000100",
    distributed_dg,
)
```

Each rank writes only its owned Maxwell degrees of freedom. The manifest is published atomically after every piece succeeds. Loading validates checksums, rank count, DG order, and owned global element IDs before refreshing ghost face traces. Restart currently requires the same rank count and partition.

13.5 ParaView time series

The distributed Poisson-bracket experiment writes owned elements only, preserves global element IDs and owner ranks, and represents DG discontinuities with element-local points. Use:

```
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --paraview-every=10
```

Open `fields.pvd` in ParaView. It indexes `fields/fields_stepNNNNNNNN.pvtu`. The periodic driver writes each element as a VTK Lagrange tetrahedron containing every DG node, so orders $N > 1$ are not reduced to a four-corner linear visualization. Its rank-local VTU pieces contain the numerical, exact, and error vectors for E and H, their magnitudes, global element IDs, owner rank, polynomial order, and physical time.

13.6 Run I/O organization

The production Maxwell drivers treat each `-output-dir` as one isolated run directory. A run directory is intended to be a complete scientific record: it contains the resolved configuration, input provenance, diagnostics, visualization files, checkpoint data, and status metadata for one execution. The current canonical layout is:

```
RUN_DIR/
  config/
    run_layout.toml
    resolved_config.toml
    run_metadata.toml
    partition_metadata.csv
  input/
    input_manifest.toml
  diagnostics/
    energy.csv
    quadrature_diagnostics.csv
```

```
    integration_points_rankNNNN.csv
  fields/
    fields_stepNNNNNNNN.pvtu
    rank-local VTU pieces
  checkpoints/
    stepNNNNNNNN/
    latest.toml
  logs/
  plots/
  tables/
  fields.pvd
  paraview_series.csv
  status.toml
```

The `logs/`, `plots/`, and `tables/` directories are created by the run-layout layer even when the production driver itself does not write into them. They provide stable locations for scheduler logs, post-processing figures, and derived tables.

For compatibility with older plotting and post-processing scripts, the run root also contains symlinks:

```
RUN_DIR/energy.csv -> diagnostics/energy.csv
RUN_DIR/quadrature_diagnostics.csv -> diagnostics/quadrature_diagnostics.csv
RUN_DIR/run_metadata.toml -> config/run_metadata.toml
RUN_DIR/partition_metadata.csv -> config/partition_metadata.csv
```

New tools should prefer the canonical paths in `diagnostics/` and `config/`. Older scripts that read the root-level CSV and TOML filenames continue to work through the symlinks.

13.6.1 Automatic isolated run directories

When `-output-dir` is supplied, the driver writes exactly there. For routine numerical experiments, prefer `-run-root`. This creates a timestamped subdirectory automatically:

```
mpiexec -n 4 julia --project=. \
  examples/distributed_periodic_poisson_bracket_maxwell.jl \
  --run-root=output/runs \
  --run-name=periodic-cuboid-af-p4m1-r4 \
  --partitions=4 \
  --flux=alternating \
  --order=4 --esprk-order=5
```

The resulting directory has the form

```
output/runs/20260708T153000_periodic-cuboid-af-p4m1-r4/
```

where the timestamp is generated once on rank zero and broadcast to all MPI ranks. The run name is sanitized to lower-case path-safe text. If `-run-name` is omitted, the driver constructs a name from the problem class, flux, polynomial order, and MPI rank count.

`-run-root` and `-output-dir` are intentionally mutually exclusive. Use `-output-dir` when an exact path is required, and use `-run-root` when a study launches many runs and each run should be isolated automatically.

13.6.2 Configuration and input provenance

`config/resolved_config.toml` stores the fully resolved driver configuration, including defaults that were not explicitly typed on the CLI. `input/input_manifest.toml` records the mesh path,

partition path, distributed mesh cache, checkpoint directory, restart checkpoint, and mesh-load mode used by the run. `config/run_layout.toml` records the resolved directory paths themselves.

`status.toml` is a small machine-readable status file. It is first written as `prepared`, then updated to `running`, and finally to `complete`. The final status contains the final step, final time, elapsed integration time, and final checkpoint path.

13.6.3 Checkpoint paths

By default, checkpoints are written to `RUN_DIR/checkpoints/`. Supplying `-checkpoint-dir=PATH` overrides this and can place checkpoints outside the run directory. This is useful on machines with a dedicated high-throughput scratch filesystem, but it reduces the self-contained nature of the run artifact. When using an external checkpoint directory, keep the `input/input_manifest.toml` file with the run because it records the external path.

13.7 Run and partition metadata

Every production experiment writes:

- `config/run_metadata.toml`: configuration, Julia version, MPI size, time-step data, elapsed solve time, and final checkpoint;
- `config/partition_metadata.csv`: owned and ghost elements, local nodes, neighbor ranks, interface-face counts, halo-size ratios, send/receive buffer sizes, and global element ranges;
- `config/resolved_config.toml`: resolved driver configuration;
- `input/input_manifest.toml`: mesh, partition, distributed mesh cache, checkpoint, restart, and load-mode provenance;
- `status.toml`: prepared/running/complete status and final run summary;
- `paraview_series.csv`: step, time, and PVTU dataset index.

Chapter 14

Diagnostics and Troubleshooting

14.1 MPI run hangs

- Confirm every rank enters the same collective constructor and time integration sequence.
- Confirm the launch rank count matches the partition file.
- Confirm MPI.jl and `mpiexec` use compatible MPI libraries. Inspect the configured backend with `julia -project=. -e 'using MPI; MPI.versioninfo()'` and use `mpiexecjl` when the system launcher targets a different MPI implementation.
- Do not let only rank zero call `distributed_maxwell_energy`; it contains an `Allreduce`.
- Ensure all ranks use the same communication tag for a given exchange.

14.2 Wrong interface flux or rank-dependent error

- Run `test_mpi_face_permutation.jl`.
- Check tetrahedron connectivity and face orientation.
- Check that face-node generation is identical on all ranks.
- Tighten or relax `trace_tol` only after inspecting mesh scale.
- Verify that the partition includes complete face adjacency, not merely node adjacency.

14.3 Missing or incorrect boundary conditions

- Confirm the VTU file contains triangular boundary cells.
- Confirm their cell-data array is named `boundary_id`, or pass a different `boundary_tag_name`.
- Print `unique(tri_data(mesh, "boundary_id"))`.
- Register every physical PEC tag explicitly.
- Remember that tag 10 is a convention in the examples, not a hard requirement of the solver.
- For the periodic box driver, verify tags $1 \leftrightarrow 2$, $3 \leftrightarrow 4$, and $5 \leftrightarrow 6$ and matching translated triangulations.
- Periodic tags belong in the periodic exchange and should map to `MaxwellBC_None` in the physical-boundary registry.

14.4 Periodic driver fails during partitioning

- Confirm `mpmetis` is installed and visible in `PATH`.
- Confirm `-partitions` equals the `mpiexec -n` rank count.
- Use `-partitions=N` for the rank count. The singular option `-partition=PATH` is only for an existing partition file.
- If a partition contains a different number of entries than the mesh has tetrahedra, regenerate it for that mesh with `-repartition`.
- Use `-repartition` after replacing the source mesh.
- Use `-rebuild-distributed-mesh` after changing the partition.
- Use DG order two or higher for the supplied traveling wave.

14.5 Unstable time integration

- Reduce the CFL factor.
- Inspect the worst element reported by `element_size_diagnostics`.
- Check for degenerate or highly skewed tetrahedra.
- Verify positive ε and μ.
- Use the upwind Hesthaven–Warburton flux when damping unresolved jumps is acceptable.
- Confirm that the chosen RK or ESPRK order is supported.

14.6 Poisson-bracket errors

- Use `PoissonBracketFormulation()` with central flux only.
- Use the partitioned symplectic driver.
- Set `first_partition = :H`.
- Use ESPRK order one through six.

14.7 Threaded run is slower

- Increase the problem size; small meshes are overhead dominated.
- Verify the actual Julia thread count.
- Avoid oversubscribing cores in hybrid MPI/threaded runs.
- Pin ranks and threads appropriately for the machine.
- Use the benchmark harness and compare median times.

14.8 Memory use is larger than expected

- Use `build_distributed_dg_from_partition` for repeated runs, or `build_distributed_dg_from_root` during preparation.
- Remember that each rank stores owned elements plus one face halo.
- Higher polynomial order increases storage in proportion to N_p.
- Explicit RK methods store several complete right-hand sides.
- Root temporarily retains the complete global mesh and partition only while preparing rank-local mesh files.

Chapter 15

Current Limitations and Roadmap

15.1 Current limitations

- Three-dimensional tetrahedral meshes only.
- Affine straight-sided tetrahedral geometry.
- Source-free Maxwell equations.
- Material coefficients are isotropic and piecewise constant per tetrahedron; curved or nodally varying tensors are not implemented.
- The characteristic absorbing boundary is first order. It is separate from the nonlinear PML and remains useful as an outer termination.
- The nonlinear PML supports scalar homogeneous coefficients and piecewise-constant isotropic `MaxwellElementMaterials`. Anisotropic material tensors and nodally varying material coefficients are not implemented.
- The nonlinear PML is dissipative, so ESPRK runs with PML should be interpreted as the selected partitioned update applied to a damped right-hand side, not as a conservative symplectic map.
- No adaptive mesh refinement.
- No distributed mesh repartitioning during a run.
- Checkpoint restart requires the same MPI rank count and partition.
- Rank-local mesh files currently require a shared filesystem.
- Initial mesh-cache preparation still reads and partitions the complete mesh on root; subsequent runs are rank-local.
- Communication is nonblocking, but currently completed before local DG evaluation; volume work is not yet overlapped with trace exchange.
- Stored ghost elements currently pass through local volume evaluation before ghost residuals are cleared.

15.2 Recommended next development stages

1. Add impressed electric and magnetic sources.
2. Add anisotropic and within-element varying material tensors.
3. Validate nonlinear-PML reflection errors for oblique incidence, material jumps, and corner layers.
4. Add alternative higher-order absorbing boundary conditions.
5. Split the right-hand side into owned volume work and interface surface work, then overlap communication.
6. Skip ghost volume computation.
7. Add MPI strong- and weak-scaling benchmarks.
8. Add repartitioning restart and portable long-term checkpoint formats.
9. Add direct parallel ingestion from partition-aware mesh formats so cache preparation also avoids root assembly.

Appendix A

API Quick Reference

API	Purpose
`read_vtu_mesh(path)`	Read XML VTU tetrahedra, boundary triangles, and cell data into `RawVTUMesh`.
`DGDiscretization(mesh,order;...)`	Build complete serial/threaded DG operators.
`SerialBackend()`	Select scalar local kernels.
`ThreadedBackend()`	Select Julia-threaded local Maxwell kernels.
`HesthavenWarburtonFormulation(kind)`	Strong-form Maxwell DG with central or upwind flux.
`PoissonBracketFormulation()`	Partitioned weak Maxwell operator with central flux.
`MaxwellBoundaryRegistry(dict)`	Map integer boundary tags to boundary kinds.
`MaxwellMaterial(epsilon,mu)`	Define one positive isotropic material.
`maxwell\_element\_materials(...)`	Map mesh material IDs to element permittivity and permeability arrays.
`MaxwellNonlinearPML(...)`	Store nodal directional damping, nonlinear parameter a, and denominator regularization.
`polynomial\_pml\_sigma(...)`	Evaluate a one-sided polynomial damping profile.
`build\_maxwell\_nonlinear\_pml(...)`	Sample three damping functions at the local DG nodes.
`nonlinear\_pml\_source(...)`	Evaluate the six nonlinear PML source components at one point.
`maxwell\_nonlinear\_pml\_rhs!(...)`	Assemble ordinary Maxwell terms and the serial or distributed nonlinear PML source.
`distributed\_maxwell\_nonlinear\_pml\_partitioned\_symplectic\_rk\_step!(...)`	Advance one distributed Poisson-bracket ESPRK step with nonperiodic PML boundaries.
`distributed\_periodic\_maxwell\_nonlinear\_pml\_partitioned\_symplectic\_rk\_step!(...)`	Advance one distributed Poisson-bracket ESPRK step with periodic face pairs and nonlinear PML.
`distributed\_maxwell\_nonlinear\_pml\_rk\_step!(...)`	Advance one distributed explicit RK step with nonperiodic PML boundaries.
`distributed\_periodic\_maxwell\_nonlinear\_pml\_rk\_step!(...)`	Advance one distributed explicit RK step with periodic face pairs and nonlinear PML.
`run\_maxwell\_nonlinear\_pml\_time\_steps!(...)`	Advance a serial Poisson-bracket nonlinear-PML problem with explicit RK.

API	Purpose
`run\_distributed\_maxwell\_nonlinear\_pml\_time\_steps!(...)`	Advance a distributed Poisson-bracket nonlinear-PML problem with explicit RK and global energy reporting.
`interpolate_maxwell_field(...)`	Sample analytic electric and magnetic functions at DG nodes.
`estimate_maxwell_dt(...)`	Compute the volume-to-surface CFL estimate.
`maxwell_energy(...)`	Compute serial electromagnetic energy.
`run_maxwell_time_steps!(...)`	Advance Hesthaven–Warburton with explicit RK.
`run_maxwell_partitioned_symplectic_time_steps!(...)`	Advance a partitioned Maxwell system with ESPRK.
`read_metis_epart(path)`	Read zero-based element partition IDs.
`build_distributed_mesh(...)`	Build one rank's owned-plus-halo mesh from global arrays.
`build_distributed_dg(...)`	Build distributed DG when the global mesh is replicated.
`build_distributed_dg_from_root(...)`	Distribute rank-local DG meshes from a root-owned global mesh or root-read files.
`build\_distributed\_periodic\_maxwell\_exchange(...)`	Pair periodic boundary faces and allocate periodic trace communication.
`distributed\_maxwell\_invariants(...)`	Reduce energy, charge, linear momentum, and angular momentum over owned elements.
`prepare_distributed_mesh_partition(...)`	Write one checksummed owned-plus-halo mesh piece per rank.
`build_distributed_dg_from_partition(...)`	Build DG by independently loading one prepared mesh piece per rank.
`write_distributed_checkpoint(...)`	Atomically write owned Maxwell fields and restart metadata.
`load_distributed_checkpoint(...)`	Validate and restore a same-partition Maxwell checkpoint.
`RunPaths(root)`	Resolve the canonical run-directory tree under a root path.
`prepare_run_directory(...)`	Create the run directory, standard subdirectories, compatibility symlinks, layout metadata, and initial status.
`isolated_run_directory(...)`	Build a timestamped `STAMP_run-name` directory below a run root.
`write_resolved_run_config(...)`	Write `config/resolved_config.toml`.
`write_run_input_manifest(...)`	Write `input/input_manifest.toml`.
`collectively_write_run_provenance(...)`	Root-rank collective writer for resolved configuration and input manifest files.
`collectively_write_run_status(...)`	Root-rank collective writer for `status.toml`.
`write_distributed_run_metadata(...)`	Write TOML run metadata and CSV partition metadata.
`exchange_maxwell_ghost_traces!(...)`	Exchange six Maxwell face traces.
`distributed_maxwell_energy(...)`	Compute owned-only local energy followed by a global sum.
`run_distributed_maxwell_time_steps!(...)`	Distributed explicit RK driver.

API	Purpose
`run_distributed_maxwell_partitioned_symplectic_time_steps!(...)`	Distributed Poisson-bracket ESPRK driver.
`write_metis_mesh_from_vtk(...)`	Convert a supported legacy VTK tetra mesh to METIS format.
`write_partition_vtk(...)`	Write a legacy VTK mesh colored by partition.
`write_partition_piece_vtks(...)`	Write owned-element mesh pieces.

Appendix B

Command Quick Reference

```
# Instantiate
julia --project=. -e 'using Pkg; Pkg.instantiate()'

# Package tests
julia --project=. -e 'using Pkg; Pkg.test()'

# Distributed Maxwell demo
mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl

# Distributed nonlinear PML pulse
mpiexec -n 2 julia --project=. \
  examples/distributed_nonlinear_pml_maxwell.jl \
  --nx=8 --order=2 --final-time=1.3 \
  --pml-width=0.5 --sigma-max=12

# Production Poisson-bracket PML reflection sweep
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_pml_validation.jl \
  --mesh=examples/meshes/periodic_box_structured_nx16_ny8_nz8.vtk \
  --central-wavelength=0.5 \
  --fluxes=centered,alternating \
  --pml-widths=0.25,0.5 --sigma-maxes=8,12 \
  --sigma-degrees=2,3 --paraview-every=25

# Production Poisson-bracket run with time-series output and checkpoints
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --paraview-every=10 --checkpoint-every=50

# Production run in an isolated timestamped directory
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --run-root=output/runs \
  --run-name=cavity-pec-centered-p2-r2 \
  --paraview-every=10 --checkpoint-every=50

# PMC Poisson-bracket cavity
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc

# Poisson-bracket cavity with six-sided nonlinear PML
mpiexec -n 2 julia --project=. \
```

```
  examples/distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc \
  --pml-width=0.2 --pml-sigma-max=12 --esprk-order=4

# Distributed PMC convergence study
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_poisson_bracket_maxwell.jl \
  --boundary-condition=pmc \
  --output=output/convergence_distributed_poisson_bracket_pmc.csv

# Distributed unstructured cavity convergence study
mpiexec -n 2 julia --project=. \
  examples/convergence_distributed_poisson_bracket_maxwell.jl \
  --mesh-family=unstructured \
  --nx-targets=2,4,8,16 \
  --orders=2,3,4 \
  --periods=1 \
  --output=output/convergence_distributed_poisson_bracket_unstructured.csv

# Restart to an absolute target time
mpiexec -n 2 julia --project=. \
  examples/distributed_poisson_bracket_maxwell.jl \
  --restart=output/distributed_poisson_bracket/checkpoints/step00000050 \
  --final-time=0.5

# Plot diagnostics from the canonical diagnostics subdirectory
python3 examples/plot_maxwell_energy.py \
  output/distributed_poisson_bracket/diagnostics/quadrature_diagnostics.csv

# MPI tests
mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl

# Mesh inspection
julia --project=. examples/solver/read_vtu_mesh.jl mesh.vtu

# METIS preparation and partitioning
julia --project=. examples/write_metis.jl
mpmetis examples/meshes/tet_mesh.mesh 4

# Partition visualization
julia --project=. examples/write_partition_vtk.jl

# Serial stationary case
julia --project=. examples/solver/stationary_pec_cube.jl --help

# Threaded Poisson-bracket case
julia --project=. --threads=8 \
  examples/solver/stationary_pec_cube_pb_esprk.jl --help

# Backend benchmark
julia --project=. --threads=8 \
  benchmark/solver/benchmark_backends.jl --help

```

```
# Thread scaling
THREAD_COUNTS="1 2 4 8" \
  bash benchmark/solver/run_thread_scaling.sh

# Plot benchmark data
python3 benchmark/solver/plot_thread_scaling.py

# Build this guide
pdflatex docs/DiscoGMPI_User_Guide.tex
pdflatex docs/DiscoGMPI_User_Guide.tex
```

Appendix C

Reproducibility Checklist

Record the following information with every reported result:

- `DiscoGMPI` commit hash and working-tree status;
- Julia version and project/manifest state;
- MPI implementation and version;
- number of MPI ranks and Julia threads per rank;
- CPU/node allocation and affinity settings;
- mesh file, tetrahedron count, boundary tags, and partition file;
- polynomial order N, N_p, and N_{fp};
- formulation and flux;
- RK or ESPRK order and partition order;
- ε, μ, CFL, Δt, and number of steps;
- PML interface locations, $\sigma_{\max}$, profile degree, a, and denominator regularization when nonlinear PML damping is active;
- initial condition and boundary registry;
- initial/final energy and error norms;
- output and benchmark methodology.

License

`DiscoGMPI` is distributed under the MIT License. See `LICENSE.md` in the repository for the complete license text.